☀ GM-SUNSHINE.COM



POSTGRESQL

PostgreSQL

100 Interview Questions

QUESTIONS & ANSWERS

Save it before your next interview · Follow for more

From SELECT to MVCC, the 100 that matter

PostgreSQL has become the default database for new projects, and interviews have followed: they now reach well past basic SQL into indexing strategy, the query planner, MVCC, and transaction isolation. This guide covers that whole range, from the fundamentals to the internals that separate someone who *uses* Postgres from someone who truly understands it.

Every answer explains the reasoning a strong candidate gives, with runnable SQL and the trade-offs behind each choice. Read it once cover to cover, then use it the night before to reload the map.

HOW TO USE THIS GUIDE

- **Run the queries.** Spin up a local Postgres and actually execute the examples, especially the `EXPLAIN ANALYZE` ones.
- **Explain, do not memorize.** After each answer, say the why in your own words. That is what interviews reward.
- **Spend your last hour** on indexing, the query planner, and MVCC and isolation. That is where senior interviews are won.

What's inside

Part I	Fundamentals — data types, DDL, constraints, CRUD, aggregates, GROUP BY, NULL handling	Q1–20
Part II	Joins & Queries — join types, subqueries, CTEs, window functions, upserts, set operations	Q21–40
Part III	Indexing & Performance — B-tree, EXPLAIN ANALYZE, index types, VACUUM, query planning	Q41–60
Part IV	Transactions & Concurrency — ACID, isolation levels, MVCC, locks, deadlocks, SKIP LOCKED	Q61–80
Part V	Admin, JSON & Advanced — roles, backups, replication, partitioning, JSONB, full-text, PL/pgSQL	Q81–100

1 What PostgreSQL Is

FUNDAMENTALS

Q What is PostgreSQL and what sets it apart from other databases?

A PostgreSQL is a free, open-source **object-relational** database management system (ORDBMS) with more than 35 years of active development. It stores data in relational tables but also embraces object-oriented concepts such as table inheritance, composite types, and user-defined types, which is why it is called object-relational rather than purely relational.

Its defining traits are extensibility and standards compliance. You can add your own data types, operators, index methods, aggregate functions, and even procedural languages (PL/pgSQL, PL/Python) without touching the core. It closely tracks the ANSI SQL standard while adding rich features like window functions, CTEs, `JSONB`, and full-text search.

PostgreSQL is fully ACID-compliant and uses **MVCC** (Multi-Version Concurrency Control) so readers never block writers and writers never block readers. This makes it a strong default for both transactional (OLTP) and analytical workloads.

```
SELECT version();
-- PostgreSQL 16.x on x86_64-pc-linux-gnu ...
```

> **GOTCHA**
>
> People say "Postgres" and "PostgreSQL" interchangeably; both are correct. Do not confuse it with "PostgreSQL is just MySQL with a different name" — the concurrency model (MVCC vs locking) and extensibility differ substantially.

2 Core Data Types

FUNDAMENTALS

Q What are the key data types in PostgreSQL?

A PostgreSQL ships with a rich type system. Choosing the right type is a WHY question: it enforces correctness at the storage layer and lets the planner pick efficient operations. Picking `TEXT` for a number, for example, throws away range checks and index efficiency.

Common numeric types are `SMALLINT`, `INTEGER`, `BIGINT`, `NUMERIC(p,s)` for exact decimals (money!), and `REAL` / `DOUBLE PRECISION` for floats. For text use `TEXT` or `VARCHAR(n)`; for truth values `BOOLEAN`.

Date/time uses `TIMESTAMPTZ` (timezone-aware, preferred), `DATE`, and `INTERVAL`. PostgreSQL also has first-class **arrays**, `JSONB` (binary JSON with indexing), `UUID`, `ENUM`, network types (`INET`), and range types.

```
CREATE TABLE event (
  id       BIGINT,
  amount   NUMERIC(12,2),
  tags     TEXT[],
  payload  JSONB,
  ref      UUID,
  at       TIMESTAMPTZ
);
```

> **GOTCHA**
>
> Never use `FLOAT` / `DOUBLE PRECISION` for money — **0.1 + 0.2 ≠ 0.3** in binary floating point. Use `NUMERIC` for exact decimal arithmetic.

3 Auto-Generated Keys

FUNDAMENTALS

Q SERIAL vs IDENTITY vs UUID for primary keys?

A All three give you unique keys, but they differ in mechanics and semantics. `SERIAL` is not a real type — it is syntactic sugar that creates an `INTEGER` column plus an owned sequence and a `DEFAULT nextval(...)`. It predates the SQL standard identity feature.

`GENERATED ... AS IDENTITY` is the SQL-standard replacement and is now recommended. `GENERATED ALWAYS` prevents callers from inserting an explicit value (protecting the sequence), while `BY DEFAULT` allows overrides. It cleans up several SERIAL footguns around ownership and permissions.

`UUID` keys are 128-bit, globally unique, and can be generated on the client without a round trip — great for distributed systems and hiding row counts. The trade-off: random UUIDs (v4) hurt B-tree locality and index bloat; consider UUIDv7 or `BIGINT` identity when insert throughput matters.

```
CREATE TABLE users (
  id  BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  ref UUID DEFAULT gen_random_uuid()
);
```

> **GOTCHA**
>
> Prefer `BIGINT` over `INT` for identity keys — a 32-bit sequence exhausts at ~2.1 billion rows, and migrating a live PK later is painful.

4 Column Constraints

FUNDAMENTALS

Q How do you create a table with column constraints?

A Constraints move data-integrity rules into the database so they hold no matter which application or script writes the data. This is the WHY: application-only validation is bypassable, but a constraint is enforced by the engine on every write.

Key column constraints are `NOT NULL` (value required), `UNIQUE` (no duplicates, backed by an index), `CHECK` (arbitrary boolean expression), and `DEFAULT` (value used when none supplied). `PRIMARY KEY` is effectively `UNIQUE` + `NOT NULL`.

Constraints can be named for clearer error messages and easier `ALTER TABLE ... DROP CONSTRAINT`. A `CHECK` can reference multiple columns of the same row when written at the table level.

```
CREATE TABLE product (
  id     BIGINT GENERATED ALWAYS AS IDENTITY,
  sku    TEXT NOT NULL UNIQUE,
  price  NUMERIC(10,2) NOT NULL CHECK (price > 0),
  active BOOLEAN NOT NULL DEFAULT true
);
```

> **GOTCHA**
>
> A `CHECK` constraint passes when the expression is **TRUE or NULL** (unknown). `CHECK (price > 0)` does NOT reject a NULL price — you need a separate `NOT NULL`.

5 Keys and Referential Integrity

FUNDAMENTALS

Q Primary keys vs foreign keys and referential integrity?

A A **primary key** uniquely identifies each row in a table and cannot be NULL. A **foreign key** is a column (or set) in one table that references the primary (or unique) key of another, modeling a relationship between rows.

Referential integrity is the guarantee that a foreign key always points to a row that actually exists. PostgreSQL enforces this: you cannot insert an `order` whose `customer_id` has no matching customer, and you cannot delete a customer that still has orders unless you specify a referential action.

Those actions — `ON DELETE` / `ON UPDATE` with `CASCADE`, `SET NULL`, `SET DEFAULT`, `RESTRICT`, or `NO ACTION` — decide what happens to children when a parent changes. Always index the FK column; PostgreSQL does not create that index automatically.

```
CREATE TABLE orders (
  id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  customer_id BIGINT NOT NULL
    REFERENCES customers(id) ON DELETE CASCADE
);
```

> **GOTCHA**
>
> Postgres auto-indexes primary and unique keys but **not** foreign keys. A missing FK index makes parent deletes and joins slow — add it yourself.

6 NULL and Three-Valued Logic

FUNDAMENTALS

Q What are NULL semantics and three-valued logic?

A NULL means "unknown" or "not applicable," not zero and not an empty string. Because a value can be unknown, SQL uses **three-valued logic**: expressions evaluate to TRUE, FALSE, or UNKNOWN. This is the WHY behind many surprising query results.

Any comparison with NULL yields UNKNOWN, so `NULL = NULL` is not TRUE. You must test with `IS NULL` / `IS NOT NULL`. In a `WHERE` clause only rows that evaluate to TRUE are returned, so UNKNOWN rows are filtered out.

NULL also propagates through arithmetic (`5 + NULL` is NULL) and affects aggregates, uniqueness, and joins. In a `UNIQUE` constraint, multiple NULLs are allowed by default because they are considered distinct (unless you add `NULLS NOT DISTINCT`, available since PG 15).

```
SELECT NULL = NULL   AS eq,      -- NULL (unknown)
       NULL IS NULL  AS is_null; -- true
```

> **GOTCHA**
>
> `WHERE col NOT IN (1, 2, NULL)` returns **no rows** because the NULL makes the whole predicate UNKNOWN. Prefer `NOT EXISTS` or filter NULLs out first.

7 COALESCE and NULLIF

FUNDAMENTALS

Q How do COALESCE and NULLIF help handle NULLs?

A `COALESCE(a, b, c, ...)` returns the first non-NULL argument, so it is the standard way to supply a fallback/default value. It short-circuits, evaluating arguments left to right and stopping at the first non-NULL.

`NULLIF(a, b)` returns NULL if `a = b`, otherwise `a`. It is the inverse tool: it converts a specific sentinel value into NULL. A classic use is `NULLIF(divisor, 0)` to avoid division-by-zero errors.

Both are ANSI SQL and part of the broader NULL toolkit that includes `GREATEST` / `LEAST` (which ignore NULLs) and the `IS DISTINCT FROM` operator, which treats NULL as a comparable value so `NULL IS DISTINCT FROM 1` is TRUE.

```
SELECT COALESCE(nickname, first_name, 'Anonymous') AS display,
       revenue / NULLIF(orders, 0) AS avg_order
FROM customer;
```

> **GOTCHA**
>
> All `COALESCE` arguments must be type-compatible. Mixing `INTEGER` and `TEXT` without a cast raises an error, and the result type is the resolved common type.

8 Filtering with WHERE

FUNDAMENTALS

Q How does basic SELECT filtering with WHERE work?

A The `WHERE` clause filters rows before grouping and projection, keeping only rows for which the predicate is TRUE. Understanding evaluation order (FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY) explains why you cannot reference a `SELECT` alias in `WHERE`.

Comparison operators are `=`, `<>` (or `!=`), `<`, `>`, `<=`, `>=`. Logical operators `AND`, `OR`, `NOT` combine predicates, with `AND` binding tighter than `OR` — use parentheses to be explicit.

Convenience predicates include `BETWEEN` (inclusive on both ends), `IN (...)`, and `IS NULL`. A **sargable** predicate (one that can use an index) keeps the column bare on the left: `created_at >= '2026-01-01'` can use an index, whereas wrapping the column in a function often cannot.

```
SELECT id, email
FROM users
WHERE age >= 18
  AND (country = 'US' OR country = 'CA')
  AND deleted_at IS NULL;
```

> **GOTCHA**
>
> `age BETWEEN 18 AND 65` is inclusive of both 18 and 65. If you mean an exclusive upper bound, write `age >= 18 AND age < 65`.

9 Sorting and Pagination

FUNDAMENTALS

Q How do ORDER BY, LIMIT, and OFFSET work (and keyset pagination)?

A `ORDER BY` sorts the result set; without it, row order is **not guaranteed** in PostgreSQL (MVCC and parallel scans can return rows in any physical order). You can sort by column, expression, or output position, with `ASC` / `DESC` and `NULLS FIRST` / `NULLS LAST`.

`LIMIT n` caps the number of rows returned; `OFFSET m` skips the first m. Together they implement classic pagination. But OFFSET still scans and discards the skipped rows, so page 10,000 gets progressively slower.

Keyset (cursor) pagination avoids that cost by remembering the last row's sort key and using a `WHERE` comparison instead of OFFSET. It stays fast at any depth and is stable under concurrent inserts.

```
-- Keyset: next page after last seen (created_at, id)
SELECT id, title, created_at
FROM posts
WHERE (created_at, id) < ('2026-07-01', 500)
ORDER BY created_at DESC, id DESC
LIMIT 20;
```

> **GOTCHA**
>
> Always include a unique tiebreaker (like `id`) in `ORDER BY` for pagination. Sorting by a non-unique column alone gives unstable ordering, so rows can be skipped or repeated across pages.

10 DISTINCT and DISTINCT ON

FUNDAMENTALS

Q What is the difference between DISTINCT and DISTINCT ON?

A `SELECT DISTINCT` removes duplicate rows from the result, comparing every selected column. It effectively groups by all output columns and returns one row per unique combination.

`DISTINCT ON (expr)` is a PostgreSQL extension that returns the first row for each distinct value of the given expression(s). "First" is determined by the `ORDER BY`, so the leading `ORDER BY` columns must match the `DISTINCT ON` columns. This is a compact way to get the latest/top row per group.

The WHY: many databases force you to write a window-function or self-join to get "one row per group." `DISTINCT ON` does it in a single clause and is often faster because it can stop after the first qualifying row per group.

```
-- Latest login per user
SELECT DISTINCT ON (user_id) user_id, logged_at, ip
FROM logins
ORDER BY user_id, logged_at DESC;
```

> **GOTCHA**
>
> With `DISTINCT ON`, the `ORDER BY` must **start** with the same expressions, or Postgres raises an error / returns arbitrary rows. The remaining sort keys pick which row "wins."

11 Aggregate Functions

FUNDAMENTALS

Q What are the aggregate functions, and how does COUNT(*) differ from COUNT(col)?

A Aggregates collapse many rows into one summary value: `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`, plus statistical ones like `STDDEV` and array/string builders such as `array_agg` and `string_agg`.

The key distinction: `COUNT(*)` counts **all rows**, including those with NULLs. `COUNT(col)` counts only rows where `col` is NOT NULL. `COUNT(DISTINCT col)` counts distinct non-NULL values. This directly follows from NULL semantics.

Aggregates ignore NULL inputs (except `COUNT(*)`). So `AVG(col)` divides the sum of non-NULL values by the count of non-NULL values — NULLs are not treated as zero, which changes the average versus `COALESCE(col,0)`.

```
SELECT COUNT(*)          AS rows,
       COUNT(phone)      AS with_phone,
       COUNT(DISTINCT city) AS cities,
       AVG(age)          AS avg_age
FROM users;
```

> **GOTCHA**
>
> `AVG` over an all-NULL set returns NULL, not 0. And `SUM` of no rows is NULL, not 0 — wrap in `COALESCE(SUM(x), 0)` if you need a numeric zero.

12 GROUP BY Rules

FUNDAMENTALS

Q How does GROUP BY work and what are its rules?

A `GROUP BY` partitions rows into groups that share the same values for the grouping expressions, then computes one aggregate row per group. It is what turns "all users" into "count of users per country."

The core rule: every column in the `SELECT` list must either appear in the `GROUP BY` or be wrapped in an aggregate. PostgreSQL enforces this strictly (unlike older MySQL). The one exception is that columns functionally dependent on a grouped primary key are allowed.

Postgres also supports `GROUPING SETS`, `ROLLUP`, and `CUBE` for multiple grouping levels in one pass — useful for subtotals and cross-tabs. You can group by expressions and by output column position too.

```
SELECT country, COUNT(*) AS n, AVG(age) AS avg_age
FROM users
GROUP BY country
ORDER BY n DESC;
```

> **GOTCHA**
>
> "column must appear in the GROUP BY clause or be used in an aggregate function" is the classic error. Add the column to `GROUP BY` or aggregate it — don't just silently pick an arbitrary value.

13 HAVING vs WHERE

FUNDAMENTALS

Q What is the difference between HAVING and WHERE?

A The distinction is about **timing in the pipeline**. `WHERE` filters individual rows **before** grouping and cannot reference aggregates. `HAVING` filters groups **after** `GROUP BY` and aggregation, so it can reference aggregate results.

Because of this order, you cannot put `SUM(amount) > 1000` in a `WHERE` clause — the sum does not exist yet. You put row-level conditions in `WHERE` and group-level conditions in `HAVING`.

Performance matters: filter as much as possible in `WHERE` so fewer rows enter the (relatively expensive) grouping step. Reserve `HAVING` for conditions that genuinely depend on the aggregate.

```
SELECT customer_id, SUM(amount) AS total
FROM orders
WHERE status = 'paid'          -- row filter, before grouping
GROUP BY customer_id
HAVING SUM(amount) > 1000;     -- group filter, after
```

> **GOTCHA**
>
> Don't move a plain row condition (like `status = 'paid'`) into `HAVING`. It still works but forces every row through grouping first, wasting work.

14 INSERT, UPDATE, DELETE, RETURNING

FUNDAMENTALS

Q What are the basics of INSERT, UPDATE, DELETE, and RETURNING?

A These are the data-modification (DML) statements. `INSERT` adds rows, `UPDATE` changes existing rows matching a `WHERE`, and `DELETE` removes matching rows. Omitting `WHERE` on UPDATE/DELETE affects the entire table.

PostgreSQL's **RETURNING** clause is a standout feature: any of these statements can return columns from the affected rows in the same round trip. This lets you grab a generated identity key or the pre/post values without a second query.

`INSERT` also supports **upsert** via `ON CONFLICT (...) DO UPDATE / DO NOTHING`, and multi-row inserts with a single `VALUES` list for efficiency. All DML runs inside a transaction, so it is atomic.

```
INSERT INTO users (email, name)
VALUES ('a@x.com', 'Ann')
ON CONFLICT (email) DO UPDATE SET name = EXCLUDED.name
RETURNING id, created_at;
```

> **GOTCHA**
>
> An `UPDATE` or `DELETE` with no `WHERE` hits every row. In `psql`, wrap risky changes in `BEGIN; ... ROLLBACK;` to preview the affected count first.

15 Strings and Pattern Matching

FUNDAMENTALS

Q How do string functions and pattern matching (LIKE, ILIKE, regex) work?

A PostgreSQL has a deep string toolkit: `length`, `lower` / `upper`, `trim`, `substring`, `split_part`, `concat`, and the `||` concatenation operator. String indexing is 1-based.

For pattern matching, `LIKE` uses `%` (any sequence) and `_` (single char) and is case-sensitive. `ILIKE` is the case-insensitive variant, a Postgres extension. Both differ from full regular expressions.

PostgreSQL also supports POSIX regex operators: `~` (match), `~*` (case-insensitive match), and their negations `!~`, `!~*`. For fast prefix/substring search at scale, use a `pg_trgm` GIN index or full-text search rather than leading-wildcard LIKE.

```
SELECT email
FROM users
WHERE email ILIKE '%@gmail.com'
   OR name ~* '^dr\.?\s';   -- regex, case-insensitive
```

> **GOTCHA**
>
> `col LIKE '%term%'` with a leading `%` cannot use a normal B-tree index — it forces a sequential scan. Use a `pg_trgm` GIN index or full-text search for substring queries.

16 Date and Time Handling

FUNDAMENTALS

Q How does PostgreSQL handle dates, times, and time zones?

A Use `TIMESTAMPTZ` (timestamp with time zone) for almost all instants. Despite the name, it does **not** store a time zone; it stores a UTC instant and converts to/from your session's `TimeZone` on input and output. This is the WHY it "just works" across zones.

`NOW()` (and `CURRENT_TIMESTAMP`) returns the transaction start time as `TIMESTAMPTZ`; use `clock_timestamp()` for the actual wall clock. `INTERVAL` represents durations, so you can do date arithmetic directly.

`AT TIME ZONE` converts between zoned and naive timestamps. Applied to a `TIMESTAMPTZ` it yields the local wall-clock time in that zone; applied to a naive `TIMESTAMP` it attaches a zone. Functions like `date_trunc`, `extract`, and `age` round out the toolkit.

```
SELECT NOW() AT TIME ZONE 'America/New_York' AS ny_local,
       NOW() - INTERVAL '7 days'             AS a_week_ago,
       date_trunc('month', NOW())             AS month_start;
```

> **GOTCHA**
>
> Avoid plain `TIMESTAMP` (without time zone) for real events — it silently ignores offsets, so the same value means different instants depending on who reads it. Store UTC in `TIMESTAMPTZ`.

17 Schemas and search_path

FUNDAMENTALS

Q What are schemas and how does search_path work?

A A schema is a namespace inside a database that holds tables, views, functions, and types. Schemas let you organize objects, separate tenants or environments, and avoid name collisions — two schemas can each have a `users` table. Every database starts with a `public` schema.

The `search_path` is an ordered list of schemas Postgres consults when you reference an object without qualifying it. The first schema containing a matching name wins, and unqualified `CREATE` lands in the first writable schema in the path.

You can fully qualify a name as `schema.table` to bypass the search path entirely. Managing the path is also a security concern: a malicious schema early in the path could shadow trusted objects.

```
SHOW search_path;              -- "$user", public
SET search_path TO app, public;
SELECT * FROM app.users;      -- fully qualified
```

> **GOTCHA**
>
> For `SECURITY DEFINER` functions, set an explicit `search_path` (e.g. `SET search_path = pg_catalog, pg_temp`) to prevent search-path hijacking attacks.

18 TEXT vs VARCHAR vs CHAR

FUNDAMENTALS

Q What is the difference between TEXT, VARCHAR, and CHAR — and why is TEXT usually fine?

A All three store character strings using the same underlying storage. The difference is length handling: `CHAR(n)` is blank-padded to a fixed length, `VARCHAR(n)` enforces a maximum length, and `TEXT` is unbounded.

In PostgreSQL there is **no performance penalty** for using `TEXT` over `VARCHAR(n)` — internally they use the same varlena representation. The length limit on `VARCHAR(n)` is just a constraint check. This is a Postgres-specific fact that surprises people coming from other engines.

So the guidance is: use `TEXT` by default and add an explicit `CHECK (length(x) <= n)` if you truly need a bound (it is easier to change than altering a type). Avoid `CHAR(n)` almost always — its blank padding causes subtle comparison and trimming surprises.

```
CREATE TABLE note (
  body TEXT NOT NULL,
  code TEXT CHECK (length(code) <= 10)  -- bound via CHECK
);
```

> **GOTCHA**
>
> `CHAR(n)` pads with trailing spaces, so `'ab'` stored in `CHAR(5)` compares oddly and `length()` may not be what you expect. Prefer `TEXT`.

19 Type Casting

FUNDAMENTALS

Q How does type casting work with :: and CAST?

A Casting converts a value from one type to another. PostgreSQL offers two syntaxes: the SQL-standard `CAST(expr AS type)` and the Postgres shorthand `expr::type`. They are equivalent; the `::` form is idiomatic in Postgres.

Casts can be implicit, assignment, or explicit depending on the type pair. Postgres deliberately removed many implicit casts to avoid silent surprises, so you sometimes must cast explicitly — for example turning a text parameter into an integer, or a string literal into `JSONB`.

Casting is also how you control the type of a literal (`'2026-07-21'::date`) and how you write down numeric precision (`amount::numeric(10,2)`). A failed cast (like `'abc'::int`) raises an error rather than returning NULL.

```
SELECT '42'::int + 1              AS n,       -- 43
       CAST('2026-07-21' AS date)  AS d,
       '{"k":1}'::jsonb ->> 'k'   AS v;      -- '1'
```

> **GOTCHA**
>
> Because `::` binds tightly, `-5::int` parses as `-(5::int)`. Fine here, but with expressions use parentheses: `(a + b)::numeric`, not `a + b::numeric`.

20 psql Essentials

FUNDAMENTALS

Q What are the essential psql commands?

A `psql` is PostgreSQL's interactive terminal client. Beyond running SQL, it has backslash **meta-commands** that query the system catalogs for you, which is why they are indispensable for exploring a database quickly.

The most-used ones: `\l` lists databases, `\c dbname` connects to one, `\dt` lists tables, `\d tablename` describes a table (columns, types, indexes, constraints), `\dn` lists schemas, and `\df` lists functions. Add a `+` (e.g. `\dt+`) for extra detail like size.

To run SQL, type a statement ending with a semicolon and press Enter. Useful helpers include `\x` (expanded/vertical output for wide rows), `\i file.sql` (run a script), `\timing` (show query duration), and `\q` to quit.

```
-- inside psql
\l                  -- list databases
\dt                 -- list tables in search_path
\d users            -- describe the users table
SELECT count(*) FROM users;   -- a query
```

> **GOTCHA**
>
> Meta-commands like `\dt` do **not** take a semicolon and only show objects in the current `search_path`. To see another schema use `\dt app.*`.

21 INNER JOIN

JOINS

Q What does an INNER JOIN do, and what determines which rows appear in the result?

A An `INNER JOIN` returns only the rows where the join predicate evaluates to `TRUE` for a pair of rows drawn from the two tables. Conceptually PostgreSQL forms the Cartesian product of the two inputs and then keeps only pairs that satisfy the `ON` condition. Rows on either side with no matching partner are discarded, which is the defining behavior that separates INNER from OUTER joins.

The word "INNER" is optional: `JOIN` alone means `INNER JOIN`. The planner does not literally build the full product; it picks a physical strategy (nested loop, hash join, or merge join) based on cost estimates, but the logical result is identical regardless of strategy.

A subtle but important point is **NULL handling**: because `NULL = NULL` is `NULL` (not TRUE), rows whose join keys are NULL never match under an equality predicate. This is a frequent source of "missing" rows that surprises people migrating from spreadsheet mental models.

```
SELECT o.id, c.name
FROM orders o
JOIN customers c ON c.id = o.customer_id
WHERE o.total > 100;
```

> **GOTCHA**
>
> Filtering on the right table's column in `WHERE` is fine for an inner join, but if you later switch to a `LEFT JOIN` the same predicate silently converts it back into an inner join by discarding the NULL-extended rows.

22 LEFT / RIGHT OUTER JOIN

JOINS

Q Explain LEFT and RIGHT OUTER JOIN and where the join condition versus the WHERE clause belongs.

A A `LEFT OUTER JOIN` keeps every row from the left table; where no right-side row satisfies the predicate, the right columns are filled with `NULL`. This preserves the left relation in full, which is why it is the workhorse for "list all X and their optional Y" queries. A `RIGHT OUTER JOIN` is the mirror image, preserving the right table; the keyword `OUTER` is optional in both.

The **placement of predicates** matters enormously. Conditions in the `ON` clause are applied before the outer-join padding step, so they decide what counts as a match. Conditions in `WHERE` are applied after padding, filtering the combined result. Putting a right-table restriction in `WHERE` eliminates the NULL-extended rows and effectively downgrades the outer join to an inner join.

Because any RIGHT JOIN can be rewritten as a LEFT JOIN by swapping table order, teams often standardize on LEFT joins for readability. The two are semantically equivalent; there is no performance difference in PostgreSQL.

```
-- keep every customer, even those with zero orders
SELECT c.name, o.id
FROM customers c
LEFT JOIN orders o
  ON o.customer_id = c.id AND o.status = 'paid';
```

> **GOTCHA**
>
> To find rows with *no* match ("anti-join"), use a LEFT JOIN plus `WHERE right.key IS NULL`. The `IS NULL` test here is legitimate because it acts on the padded NULLs, not on data.

23 FULL OUTER JOIN and CROSS JOIN

JOINS

Q What do FULL OUTER JOIN and CROSS JOIN produce, and when would you reach for each?

A A `FULL OUTER JOIN` keeps all rows from both sides: matched pairs are combined, and unmatched rows from either table appear with NULLs padding the other side. It is essentially the union of a LEFT and a RIGHT join, and it shines for reconciliation tasks where you need to see records present in one dataset but absent from the other in a single pass.

A `CROSS JOIN` produces the full Cartesian product with no predicate at all: every row of A paired with every row of B, giving `|A| × |B|` rows. It is the correct tool for generating combinations, such as pairing every product with every size, or expanding a calendar against a set of categories.

Both can explode row counts, so watch cardinality. A CROSS JOIN of two 10,000-row tables yields 100 million rows. FULL joins cannot use a plain nested loop in the same way inner joins can; PostgreSQL typically executes them via hash or merge join strategies.

```
SELECT COALESCE(a.k, b.k) AS k, a.val, b.val
FROM left_tbl a
FULL JOIN right_tbl b ON a.k = b.k;

-- every size for every product
SELECT p.name, s.label
FROM products p CROSS JOIN sizes s;
```

> **GOTCHA**
>
> Writing two tables in the `FROM` list separated by a comma with no `WHERE` is an implicit CROSS JOIN. A forgotten join condition is the classic cause of a query that unexpectedly returns millions of rows.

24 Self Joins

JOINS

Q What is a self join and what problems does it solve?

A A self join is a join of a table to itself, made possible by giving the table two different aliases so PostgreSQL can treat each reference as a distinct relation. It is not a special join type; it is an ordinary INNER, LEFT, or other join whose two inputs happen to be the same physical table.

The canonical use case is **hierarchical or adjacency data**, such as an employees table where each row has a `manager_id` pointing to another row in the same table. Joining `employees e` to `employees m` on `e.manager_id = m.id` lets you pull an employee and their manager side by side. Self joins also answer "find pairs" questions, like customers in the same city.

Aliasing is mandatory; without it the column references would be ambiguous. For arbitrary-depth hierarchies a single self join is not enough, and you move to a recursive CTE (covered later). For pair-generation, add an inequality like `a.id < b.id` to avoid duplicate and self-paired rows.

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON m.id = e.manager_id;
```

> **GOTCHA**
>
> When finding pairs, joining on `a.city = b.city` alone yields each pair twice plus each row matched to itself. Add `a.id < b.id` to keep exactly one ordered pair.

25 USING vs ON, and NATURAL JOIN

JOINS

Q Compare USING, ON, and NATURAL JOIN. Why is NATURAL JOIN risky?

A `ON` is the general form, taking any boolean expression. `USING (col)` is shorthand for an equality join on identically named columns, and it additionally **coalesces the join columns into one**: the shared column appears only once in the output and is unqualified, so `SELECT *` does not duplicate it. This makes USING cleaner when key names match.

`NATURAL JOIN` goes further: it automatically joins on *all* columns that share a name between the two tables, with no explicit list. That implicitness is exactly why it is dangerous. If someone later adds a column like `created_at` or `status` to both tables, the natural join silently starts matching on it too, quietly changing results without any change to the query text.

Because of this fragility, NATURAL JOIN is discouraged in production code. Prefer explicit `USING` when key names align, and `ON` when they differ or when the predicate is more than simple equality.

```
-- USING collapses department_id to a single output column
SELECT department_id, e.name, d.title
FROM employees e
JOIN departments d USING (department_id);
```

> **GOTCHA**
>
> With `USING` or `NATURAL`, you cannot qualify the joined column (`e.department_id` is an error in the SELECT list); reference it bare. A NATURAL JOIN between tables with no common column names degrades silently into a CROSS JOIN.

26 Subqueries: Scalar, Row, Table

QUERIES

Q What are the three shapes of subquery and where can each appear?

A A **scalar subquery** returns exactly one row and one column, so it can stand anywhere a single value is expected: in the SELECT list, in a comparison in WHERE, or even as a column default expression. If it returns more than one row at runtime, PostgreSQL raises an error; if it returns zero rows, it yields NULL.

A **row subquery** returns a single row of several columns and is used with row constructors, e.g. `(a, b) = (SELECT x, y FROM ...)`. A **table subquery** returns an arbitrary set of rows and columns; it appears in the FROM clause (a derived table, which must be aliased) or as the operand of `IN`, `EXISTS`, and `ANY` / `ALL`.

Subqueries in FROM are evaluated as their own result set and then joined; the planner can often flatten them into the outer query. Scalar subqueries in the SELECT list, by contrast, may be evaluated per output row, which can be costly if not correlated cheaply.

```
SELECT c.name,
       (SELECT count(*) FROM orders o
        WHERE o.customer_id = c.id) AS num_orders
FROM customers c;
```

> **GOTCHA**
>
> A derived table in FROM must have an alias, even if you never reference it: `FROM (SELECT ...) AS t`. Omitting the alias is a syntax error in PostgreSQL.

27 Correlated Subqueries

QUERIES

Q What makes a subquery "correlated," and what are the performance implications?

A A correlated subquery references a column from the outer query, which means it cannot be evaluated once and reused; conceptually it must be re-run for each candidate row of the outer query because its result depends on that row's values. This is what distinguishes it from an uncorrelated subquery, which is independent and can be computed a single time.

The classic pattern is "for each row, is there a related row satisfying some condition," often written with `EXISTS`. Because the inner query is logically executed per outer row, correlated subqueries can be expensive, so an index on the correlated column is usually essential to make each inner evaluation cheap.

PostgreSQL's planner is smart: it frequently rewrites correlated `EXISTS` and `IN` subqueries into semi-joins or hash joins, avoiding literal row-by-row execution. Still, understanding the logical model helps you reason about which indexes matter and when a JOIN rewrite would be clearer.

```
-- customers who have at least one order over 500
SELECT c.name
FROM customers c
WHERE EXISTS (
  SELECT 1 FROM orders o
  WHERE o.customer_id = c.id AND o.total > 500
);
```

> **GOTCHA**
>
> Inside `EXISTS`, the select list is irrelevant; `SELECT 1`, `SELECT *`, or `SELECT NULL` all behave identically because only row existence is tested, not the projected values.

28 EXISTS vs IN vs ANY

QUERIES

Q How do EXISTS, IN, and ANY differ, especially around NULLs?

A All three test membership or existence, but they behave very differently with NULLs. `EXISTS` returns TRUE as soon as the subquery yields any row and is completely NULL-agnostic, since it only cares about row presence. That robustness makes it the safest default for "is there a related row" checks.

`IN` is equivalent to `= ANY`. The dangerous case is **NOT IN with a NULL** in the subquery result: because `x <> NULL` is unknown, `NOT IN` can return no rows at all when the list contains even one NULL. This silent emptiness is one of the most common SQL correctness bugs.

`ANY` and `ALL` generalize IN to other operators, e.g. `> ANY (subquery)` or `= ALL (...)`. Performance-wise the planner often converts `IN` and `EXISTS` to the same semi-join plan, so the choice is usually about NULL semantics and clarity rather than speed.

```
-- risky: returns nothing if any banned_id is NULL
SELECT * FROM users
WHERE id NOT IN (SELECT banned_id FROM bans);

-- safe equivalent
SELECT * FROM users u
WHERE NOT EXISTS (
  SELECT 1 FROM bans b WHERE b.banned_id = u.id);
```

> **GOTCHA**
>
> Prefer `NOT EXISTS` over `NOT IN` whenever the subquery column is nullable. `NOT EXISTS` handles NULLs correctly by construction.

29 Common Table Expressions (WITH)

QUERIES

Q What are CTEs, and how has their optimization changed in modern PostgreSQL?

A A Common Table Expression, introduced with the `WITH` keyword, names a subquery so it can be referenced by later parts of the statement, improving readability and letting you decompose a complex query into logical steps. You can chain multiple CTEs, and later ones may reference earlier ones.

Historically, PostgreSQL always treated CTEs as an **optimization fence**: each CTE was materialized into a temporary result and could not be inlined or pushed down. That behavior changed in PostgreSQL 12. Now a CTE that is referenced only once and is side-effect-free is inlined by default, letting the planner optimize across the boundary just as it would a subquery.

You can control this explicitly with `MATERIALIZED` and `NOT MATERIALIZED` hints. Use `MATERIALIZED` to force a single evaluation (useful when a CTE is expensive and referenced many times, or wraps a data-modifying statement); use `NOT MATERIALIZED` to encourage inlining.

```
WITH recent AS MATERIALIZED (
  SELECT * FROM orders WHERE created_at > now() - '7 days'::interval
)
SELECT customer_id, count(*)
FROM recent
GROUP BY customer_id;
```

> **GOTCHA**
>
> On PostgreSQL 11 and earlier, a CTE was always a fence, so people used it deliberately to block bad plans. That trick silently stops working after an upgrade to 12+; add `MATERIALIZED` explicitly if you still need the fence.

30 Recursive CTEs

QUERIES

Q How does a recursive CTE work, and what are its parts?

A A recursive CTE, written `WITH RECURSIVE`, lets a query reference its own output, which is the mechanism for traversing arbitrary-depth hierarchies and graphs such as org charts, bill-of-materials trees, or category paths. It has two parts joined by `UNION` (or `UNION ALL`): a non-recursive **anchor** term that seeds the result, and a **recursive** term that references the CTE name.

Execution proceeds iteratively: the anchor runs once to produce the initial working set; then the recursive term runs repeatedly, each time consuming the rows added in the previous step, until an iteration adds no new rows. With `UNION` (as opposed to `UNION ALL`) duplicate rows are eliminated between steps, which can also stop cycles.

Because a malformed graph can loop forever, guard against cycles: either track a visited path in an array, use the built-in `CYCLE` clause (PostgreSQL 14+), or bound depth with a counter. Adding `LIMIT` outside the CTE does not by itself stop the internal recursion.

```
WITH RECURSIVE subs AS (
  SELECT id, manager_id, name, 1 AS depth
  FROM employees WHERE id = 1          -- anchor
  UNION ALL
  SELECT e.id, e.manager_id, e.name, s.depth + 1
  FROM employees e
  JOIN subs s ON e.manager_id = s.id  -- recursive
)
SELECT * FROM subs;
```

> **GOTCHA**
>
> The recursive term may reference the CTE name only once, and aggregates, `DISTINCT`, and outer joins against the recursive reference are not allowed inside it. Data with cycles will loop indefinitely unless you add cycle detection.

31 Window Functions Intro

WINDOWS

Q What is a window function, and how does OVER with PARTITION BY differ from GROUP BY?

A A window function performs a calculation across a set of rows related to the current row, but unlike an aggregate with `GROUP BY`, it **does not collapse those rows**. Each input row is preserved in the output and gains an extra computed column. This lets you show a value alongside a group-level statistic, such as each employee's salary next to their department average.

The `OVER` clause defines the window. `PARTITION BY` divides rows into groups that the function is computed within (resetting at each partition boundary), and `ORDER BY` inside `OVER` establishes ordering for functions that are position-sensitive, like ranking or running totals. Omitting `PARTITION BY` treats the whole result as one partition.

Window functions are evaluated logically after `WHERE`, `GROUP BY`, and `HAVING`, but before the final `ORDER BY` and `LIMIT`. Because of this ordering, you cannot use a window function directly in a `WHERE` clause; wrap the query in a subquery or CTE to filter on the result.

```
SELECT name, department_id, salary,
       avg(salary) OVER (PARTITION BY department_id) AS dept_avg
FROM employees;
```

> **GOTCHA**
>
> You cannot write `WHERE row_number() OVER (...) = 1`. Window functions run after WHERE, so filter on them via an outer query or CTE.

32 ROW_NUMBER vs RANK vs DENSE_RANK

WINDOWS

Q Contrast ROW_NUMBER, RANK, and DENSE_RANK. How do they treat ties?

A All three assign an ordinal within an ordered window, but they differ in how they handle ties (rows equal under the `ORDER BY`). `ROW_NUMBER` always produces a strictly increasing, unique number and breaks ties arbitrarily; two equal rows still get different numbers. It is ideal for pagination and for picking exactly one row per group.

`RANK` gives tied rows the same rank and then **skips** the following values, leaving gaps: after two rows tied at rank 1, the next rank is 3. `DENSE_RANK` also gives ties the same rank but does **not skip**: the next rank after a tie at 1 is 2. Choose based on whether you want gaps that reflect the count of tied rows.

All are true window functions requiring an `ORDER BY` within `OVER`. Because ROW_NUMBER's tie-breaking is nondeterministic, add enough columns to the window's `ORDER BY` to make the ordering stable if reproducibility matters.

```
SELECT name, score,
       row_number() OVER w AS rn,
       rank()       OVER w AS rnk,
       dense_rank() OVER w AS drnk
FROM players
WINDOW w AS (ORDER BY score DESC);
```

> **GOTCHA**
>
> For "top 3 including ties" use `RANK` or `DENSE_RANK` filtered to `<= 3`; `ROW_NUMBER` would arbitrarily cut a tie and drop a legitimately tied competitor.

33 LAG and LEAD

WINDOWS

Q What do LAG and LEAD do, and how do you handle boundary rows?

A `LAG` and `LEAD` are offset window functions that read a value from another row relative to the current one within the ordered window: `LAG` looks backward (a previous row) and `LEAD` looks forward. They are the standard tool for period-over-period comparisons, like computing the difference between this month's revenue and last month's.

Both take up to three arguments: the expression to read, an offset (default 1), and a default value to substitute when the offset falls outside the partition. Without a default, boundary rows (the first row for LAG, the last for LEAD) return `NULL`, which is why supplying a sensible default like `0` often simplifies downstream arithmetic.

These functions respect `PARTITION BY`, so the "previous" row resets at each partition boundary rather than bleeding across groups. The window's `ORDER BY` is required, because "previous" and "next" are meaningless without a defined order.

```
SELECT month, revenue,
       revenue - lag(revenue, 1, 0) OVER (ORDER BY month)
         AS mom_change
FROM monthly_sales;
```

> **GOTCHA**
>
> The default-value argument must be type-compatible with the column. `lag(revenue, 1, 0)` works for numerics, but for a text column you would pass `''` or `NULL`, not `0`.

34 Running Totals and Frames

WINDOWS

Q How do window frames work, and why does ROWS differ from RANGE?

A When a window function has an `ORDER BY`, it also has a frame: the subset of the partition the function actually operates on for the current row. A running total, for instance, sums from the partition start up to the current row. The frame is specified with a clause like `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`.

The critical distinction is `ROWS` versus `RANGE`. `ROWS` counts physical rows, so `2 PRECEDING` means literally the two prior rows. `RANGE` works on **value ranges** of the ORDER BY key, treating all peer rows (rows with equal ordering values) as a single unit. With RANGE, tied rows all share the same running total because they fall in the same value band.

This matters for defaults too: the implicit frame when you specify `ORDER BY` but no frame clause is `RANGE UNBOUNDED PRECEDING`, which can surprise you on tied keys. For a strict per-row cumulative sum, spell out `ROWS`. Moving averages use a bounded frame like `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW`.

```
SELECT day, amount,
       sum(amount) OVER (ORDER BY day
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running,
       avg(amount) OVER (ORDER BY day
         ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS mov_avg3
FROM daily;
```

> **GOTCHA**
>
> If your ORDER BY key has duplicates and you rely on the default RANGE frame, tied rows get identical cumulative values. Switch to `ROWS` for a true row-by-row running total.

35 Set Operations

QUERIES

Q Compare UNION, UNION ALL, INTERSECT, and EXCEPT.

A Set operations combine the results of two queries that must have compatible column counts and types. `UNION` returns rows present in either query but **removes duplicates**, which requires a sort or hash to deduplicate and therefore costs extra. `UNION ALL` simply concatenates both results, keeping duplicates, and is significantly faster because it skips that work.

The rule of thumb: use `UNION ALL` unless you specifically need duplicate elimination, since the deduplication in plain `UNION` is a common, avoidable performance cost. `INTERSECT` returns only rows appearing in both queries, and `EXCEPT` returns rows in the first query that are not in the second (the set difference).

All of these treat two NULLs as equal for the purpose of matching, unlike ordinary equality. Each also has an `ALL` variant. A single trailing `ORDER BY` applies to the whole combined result and must reference output column names or positions, not table-qualified names from either branch.

```
SELECT email FROM subscribers
EXCEPT
SELECT email FROM unsubscribes
ORDER BY email;
```

> **GOTCHA**
>
> `UNION` deduplicates across the *entire* combined set, not just within each branch, so it can remove rows you did not expect to be duplicates. If you know branches are disjoint, `UNION ALL` is both correct and faster.

36 CASE Expressions

QUERIES

Q Explain CASE expressions and the difference between the searched and simple forms.

A `CASE` is SQL's conditional expression, returning a value based on the first matching condition. The **searched form** evaluates arbitrary boolean conditions (`CASE WHEN x > 10 THEN ... END`), while the **simple form** compares one expression against constants (`CASE status WHEN 'a' THEN ... END`). The searched form is strictly more general.

Evaluation is short-circuit and top-to-bottom: the first `WHEN` that is TRUE wins, and later branches are not evaluated. If no branch matches and there is no `ELSE`, the result is `NULL`. All branch results must be coercible to a common type, which PostgreSQL determines from the collection of THEN/ELSE expressions.

Because CASE is an expression, it works anywhere a value is allowed: SELECT lists, `ORDER BY`, `GROUP BY`, even inside aggregates for conditional aggregation. The short-circuit property is also useful as a guard, for example returning a default before a division that would otherwise error.

```
SELECT name,
       CASE
         WHEN score >= 90 THEN 'A'
         WHEN score >= 80 THEN 'B'
         ELSE 'C'
       END AS grade
FROM results;
```

> **GOTCHA**
>
> The simple form uses `=` semantics, so `CASE x WHEN NULL THEN ...` never matches (because `x = NULL` is unknown). To test for NULL, use the searched form with `WHEN x IS NULL`.

37 FILTER and Conditional Aggregation

QUERIES

Q What does the aggregate FILTER clause do, and how does it relate to CASE inside aggregates?

A The `FILTER (WHERE ...)` clause restricts which rows an individual aggregate consumes, letting you compute several differently-scoped aggregates in one pass over the data. For example, you can count total orders and paid orders side by side without a self-join or multiple subqueries. It is a cleaner, standard-SQL alternative to the older `CASE`-inside-aggregate idiom.

The classic pre-FILTER pattern was `count(CASE WHEN cond THEN 1 END)` or `sum(CASE WHEN cond THEN amount ELSE 0 END)`. These still work and remain common, but rely on the fact that aggregates like `count` and `sum` ignore NULLs. FILTER expresses the same intent more directly and reads better.

A meaningful difference arises with `count(*)`: `count(*) FILTER (WHERE cond)` counts qualifying rows including those with NULL columns, whereas `count(CASE WHEN cond THEN 1 END)` also works but is easier to get wrong. FILTER combines naturally with `GROUP BY` and even with window functions.

```
SELECT customer_id,
       count(*) AS total,
       count(*) FILTER (WHERE status = 'paid') AS paid,
       sum(amount) FILTER (WHERE amount > 0) AS credits
FROM orders
GROUP BY customer_id;
```

> **GOTCHA**
>
> If the FILTER predicate matches no rows for a group, `count` returns `0` but `sum` and `avg` return `NULL`, not 0. Wrap with `COALESCE` if a zero is required.

38 INSERT ... ON CONFLICT (Upsert)

QUERIES

Q How does INSERT ... ON CONFLICT implement upserts, and what is the conflict target?

A `INSERT ... ON CONFLICT` is PostgreSQL's upsert: it attempts an insert and, if that would violate a unique or exclusion constraint, either does nothing (`DO NOTHING`) or updates the existing row (`DO UPDATE`). This makes "insert or update in one atomic statement" possible without a race between a separate SELECT and INSERT.

The **conflict target** tells PostgreSQL which unique index to arbitrate on, given either as a column list `(email)` or a named constraint `ON CONSTRAINT ...`. In the `DO UPDATE` branch, the special `EXCLUDED` pseudo-table exposes the values that were proposed for insertion, so you can write `SET count = table.count + EXCLUDED.count`.

You can add a `WHERE` to the DO UPDATE to make the update conditional, skipping it when it would not change anything meaningful. Note that ON CONFLICT arbitrates only on unique/exclusion constraints, not on arbitrary conditions or foreign keys, and each affected row conflicts at most once per statement.

```
INSERT INTO counters (key, hits)
VALUES ('home', 1)
ON CONFLICT (key) DO UPDATE
  SET hits = counters.hits + EXCLUDED.hits;
```

> **GOTCHA**
>
> A single statement cannot update the same target row twice; if your `VALUES` list contains two rows with the same conflict key, PostgreSQL raises "ON CONFLICT DO UPDATE command cannot affect row a second time." Deduplicate inputs first.

39 LATERAL Joins

JOINS

Q What is a LATERAL join and what does it enable that a normal subquery cannot?

A A `LATERAL` subquery in the FROM clause may reference columns from tables that appear earlier in the same FROM list. Normally a FROM-clause subquery is evaluated independently and cannot see its siblings; `LATERAL` lifts that restriction, effectively running the subquery once per row of the preceding table, like a correlated subquery that can return multiple columns and rows.

This is the idiomatic way to express **top-N-per-group** with a per-row `LIMIT`, or to call a set-returning function with arguments taken from each row. For example, for each customer fetch their three most recent orders, something a plain join cannot bound per group.

A set-returning function in FROM is implicitly lateral, so you often do not need the keyword there. Combined with `LEFT JOIN LATERAL ... ON true`, you preserve outer rows even when the lateral subquery returns nothing, mirroring outer-join semantics.

```
SELECT c.name, o.id, o.total
FROM customers c
LEFT JOIN LATERAL (
  SELECT id, total FROM orders
  WHERE customer_id = c.id
  ORDER BY created_at DESC LIMIT 3
) o ON true;
```

> **GOTCHA**
>
> With `JOIN LATERAL` (inner), customers whose lateral subquery yields no rows disappear from the result. Use `LEFT JOIN LATERAL ... ON true` to keep them with NULL-padded order columns.

40 DISTINCT ON and generate_series

QUERIES

Q How does DISTINCT ON pick one row per group, and where does generate_series help?

A `DISTINCT ON (expr)` is a PostgreSQL extension that keeps the first row for each distinct value of the given expression(s), according to the query's `ORDER BY`. It is a concise way to solve top-1-per-group: partition conceptually by the DISTINCT ON columns, sort within, and take the leading row. This is often shorter than an equivalent `ROW_NUMBER` subquery.

The crucial rule is that the leftmost `ORDER BY` expressions must match the `DISTINCT ON` expressions; only then do you add the tiebreaker column that decides which row wins. Getting this ordering wrong yields an arbitrary, nondeterministic pick per group, which is a frequent bug.

`generate_series` is a set-returning function producing a sequence of numbers or timestamps. It is invaluable for **gap-filling**: left-join a complete series of dates against sparse data so missing days appear as zero rather than vanishing. It also builds test data and calendar dimensions on the fly.

```
-- latest order per customer
SELECT DISTINCT ON (customer_id) customer_id, id, created_at
FROM orders
ORDER BY customer_id, created_at DESC;

-- every day in July, even with no sales
SELECT d::date, count(o.id)
FROM generate_series('2026-07-01', '2026-07-31', '1 day') d
LEFT JOIN orders o ON o.created_at::date = d::date
GROUP BY d ORDER BY d;
```

GOTCHA

`DISTINCT ON` requires its expressions to be the leftmost `ORDER BY` keys. If you want the results in a different final order, wrap the DISTINCT ON query in an outer query and sort there.

41 What an index is

INDEXING

Q What is an index, and why is B-tree the default in PostgreSQL?

A n index is a separate, ordered data structure that lets the database find rows matching a predicate without scanning the whole table. It stores index-key values together with pointers (a TID, or tuple identifier: block number plus offset) into the heap. The WHY is asymmetry of work: reading a few index pages plus a few heap pages is far cheaper than reading every heap page when you only want a small slice of the rows.

PostgreSQL's default access method is `btree`, a balanced tree kept sorted on the key. Because it is ordered, a B-tree supports equality (`=`), range operators (`<`, `<=`, `>`, `>=`, `BETWEEN`), `IN`, and even `ORDER BY` / `MIN` / `MAX` because the leaves are already in sorted order.

Creating a `PRIMARY KEY` or `UNIQUE` constraint automatically builds a supporting B-tree index. Other index types (GIN, GiST, BRIN, Hash, SP-GiST) exist for workloads B-tree handles poorly, but when someone says "add an index" without qualification they mean a B-tree.

```
CREATE INDEX idx_users_email ON users (email);
-- implicitly a B-tree; equivalent to USING btree (email)
```

> **GOTCHA**
>
> An index does not change what rows exist; it only changes how fast the planner can find them. It is pure derived data and can always be dropped and rebuilt.

42 When indexes hurt

INDEXING

Q When do indexes help, and what is their cost?

A Indexes help reads that are **selective**: queries returning a small fraction of the table. The WHY is that every index is redundant, ordered storage that must be kept consistent with the table. That maintenance is the price you pay on every write.

On `INSERT` and on `UPDATE` / `DELETE` that touch indexed columns, PostgreSQL must also update each affected index. More indexes means slower writes, more WAL, and more disk. A wide table with ten indexes can spend the majority of an insert's time maintaining indexes rather than writing the row.

Indexes also consume storage (often as much as the table) and compete for shared buffers and OS cache. And an index the planner never chooses is pure overhead: it slows writes and wastes space while doing nothing for reads. Non-selective predicates (a boolean that is true for 90% of rows) usually make the planner prefer a sequential scan anyway.

```
-- Check whether an index is actually used:
SELECT indexrelname, idx_scan
FROM pg_stat_user_indexes
WHERE relname = 'users';
-- idx_scan = 0 means it has never served a query
```

> **GOTCHA**
>
> HOT (heap-only tuple) updates can skip index maintenance, but only when no indexed column changes and the new tuple fits on the same page. Updating an indexed column forfeits that optimization.

43 EXPLAIN vs EXPLAIN ANALYZE

PERFORMANCE

Q What is the difference between EXPLAIN and EXPLAIN ANALYZE?

A `EXPLAIN` shows the planner's chosen plan and its **estimated** costs and row counts without running the query. `EXPLAIN ANALYZE` actually executes the query and reports the **real** time and real row counts. The WHY of preferring ANALYZE: it lets you compare estimates against reality, and a big gap between estimated and actual rows is the single most common sign of stale statistics or a mis-estimation that leads to a bad plan.

Costs are in arbitrary planner units (roughly disk-page fetches), not milliseconds. Each node shows `cost=startup..total`, `rows`, and `width`. Under ANALYZE you also get `actual time=startup..total`, actual `rows`, and `loops`. Multiply per-loop numbers by loops for the true totals.

Use `EXPLAIN (ANALYZE, BUFFERS)` to see shared/local buffer hits and reads, which distinguishes cached from cold access. Add `SETTINGS` and `WAL` for more context.

```
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM orders WHERE customer_id = 42;

Index Scan using idx_orders_cust on orders
  (cost=0.42..8.44 rows=3) (actual time=0.021..0.030 rows=3 loops=1)
  Index Cond: (customer_id = 42)
  Buffers: shared hit=4
```

> **GOTCHA**
>
> EXPLAIN ANALYZE runs the statement. For an `UPDATE` / `DELETE` / `INSERT`, wrap it in a transaction and `ROLLBACK`, or it will modify data.

44 Scan types

PERFORMANCE

Q Explain sequential scan vs index scan vs index-only scan.

A A **sequential scan** reads every heap page in order. A **index scan** walks the index to find matching TIDs, then fetches those rows from the heap. A **index-only scan** answers the query entirely from the index, never touching the heap. The WHY behind choosing among them is fraction of rows returned plus random-vs-sequential I/O cost: sequential reads are cheap per page, so for a large fraction of the table a seq scan beats thousands of random heap fetches.

An index scan wins when the predicate is selective. But each heap fetch is a random read, so past a threshold (often a few percent of the table) the planner switches to a seq scan. A `Bitmap Index Scan` is the middle ground: it collects TIDs, sorts them by page, then does one ordered heap pass, reducing random I/O.

An index-only scan is possible only when every column the query needs is in the index and the page is marked all-visible in the visibility map (so PostgreSQL can trust the row is visible without checking the heap). Otherwise it degrades to heap fetches — visible in the plan as `Heap Fetches:`.

```
EXPLAIN SELECT email FROM users WHERE email = 'a@b.com';

Index Only Scan using idx_users_email on users
  (cost=0.28..8.29 rows=1)
  Index Cond: (email = 'a@b.com')
  Heap Fetches: 0
```

> **GOTCHA**
>
> Index-only scans depend on the visibility map, which `VACUUM` maintains. A table that is never vacuumed can show high `Heap Fetches` and lose the benefit entirely.

45 Composite indexes

INDEXING

Q How do multicolumn indexes work, and why does column order matter?

A A composite index sorts rows by the first column, then by the second within equal firsts, and so on — like a phone book ordered by last name then first name. The WHY of column order: the index is only efficiently usable for a query when the query constrains a **left-to-right prefix** of the columns. You can look someone up by last name, or by last-name-and-first-name, but not by first name alone.

So an index on `(a, b, c)` supports predicates on `a`, on `(a, b)`, and on `(a, b, c)`. A query filtering only on `b` generally cannot use it (though a slower full index scan is sometimes chosen). Put the columns used for equality first, and the range/sort column last.

Column order also enables `ORDER BY` to be satisfied by the index. An index on `(customer_id, created_at)` serves `WHERE customer_id = ? ORDER BY created_at` with no separate sort step.

```
CREATE INDEX idx_orders_cust_date
  ON orders (customer_id, created_at DESC);

-- Uses the index for both filter and ordering, no Sort node:
SELECT * FROM orders
WHERE customer_id = 42 ORDER BY created_at DESC LIMIT 10;
```

> **GOTCHA**
>
> Two single-column indexes are not equivalent to one composite index. The planner can combine them with a BitmapAnd, but that is usually slower than a well-ordered composite index.

46 Covering indexes (INCLUDE)

INDEXING

Q What is a covering index and what does INCLUDE do?

A A covering index contains all columns a query needs, so it can be answered by an index-only scan without heap access. The `INCLUDE` clause (PostgreSQL 11+) adds non-key "payload" columns stored only in the leaf pages. The WHY of INCLUDE versus just widening the key: included columns add data for coverage without bloating the searchable/sorted key or affecting uniqueness.

Consider a unique index on `email` where you frequently also read `full_name`. Making `(email, full_name)` the key would weaken the uniqueness semantics and add `full_name` to every internal tree node. Instead, keep `email` as the sole key and `INCLUDE (full_name)`.

Included columns cannot be used in the index's `Index Cond` or for ordering — they are payload only. They still make the index larger, so include only what pays off.

```
CREATE UNIQUE INDEX idx_users_email_inc
  ON users (email) INCLUDE (full_name);

-- Index Only Scan; full_name comes from the index leaf:
SELECT email, full_name FROM users WHERE email = 'a@b.com';
```

> **GOTCHA**
>
> Coverage still requires the visibility map to mark pages all-visible. A heavily updated table may fall back to heap fetches even with a perfect covering index.

47 Partial indexes

INDEXING

Q What is a partial index and when is it a big win?

A A partial index indexes only the rows matching a `WHERE` predicate. The WHY: if your queries only ever care about a small subset of rows, you can index just that subset, producing a far smaller index that is cheaper to maintain and faster to scan.

The classic case is a skewed boolean or status column. In a table where 99% of orders are `shipped` and you only query `pending` ones, a partial index `WHERE status = 'pending'` indexes 1% of rows. It is tiny, hot in cache, and only touched by inserts/updates involving pending orders.

For the planner to use a partial index, its predicate must be provably implied by the query's `WHERE` clause. Partial indexes also enforce conditional uniqueness — for example, "at most one active row per user."

```
CREATE INDEX idx_orders_pending
  ON orders (created_at)
  WHERE status = 'pending';

-- Conditional uniqueness: one active subscription per user
CREATE UNIQUE INDEX uq_active_sub
  ON subscriptions (user_id) WHERE active;
```

> **GOTCHA**
>
> If a query's predicate uses a parameter (`status = $1`) the planner usually cannot prove it matches `status = 'pending'` , so the partial index is skipped. The constant must be visible at plan time.

48 Expression indexes

INDEXING

Q What is an expression (functional) index?

A An expression index stores the result of an expression rather than a raw column. The WHY: a query that filters on `lower(email)` cannot use a plain index on `email`, because the index stores the original mixed-case values. Indexing the expression itself makes the transformed value directly searchable.

The rule is that the index expression must match the query expression. If you write `CREATE INDEX ... ON users (lower(email))`, then `WHERE lower(email) = 'a@b.com'` can use it. The expression must be `IMMUTABLE` — its result must depend only on its inputs (so `now()` or a non-immutable cast is not allowed).

Expression indexes also power case-insensitive uniqueness, indexing a JSONB field, or precomputing an expensive derived value so it is never recomputed per row at query time.

```
CREATE INDEX idx_users_lower_email
  ON users (lower(email));

-- Matches the indexed expression exactly:
SELECT * FROM users WHERE lower(email) = 'a@b.com';
```

> **GOTCHA**
>
> PostgreSQL builds column statistics for the expression too, but only after `ANALYZE` runs. Right after creating an expression index, estimates for that expression may be poor until the next analyze.

49 GIN indexes

INDEXING

Q What is a GIN index and when do you use it?

A GIN (Generalized Inverted Index) is built for columns holding **composite values** where you search for elements *inside* the value: array members, JSONB keys/values, full-text lexemes, trigrams. The WHY: like a book's index of terms, GIN maps each contained element to the list of rows containing it, so "which rows contain X" is a direct lookup instead of a scan.

For each key, GIN stores a compressed posting list of matching TIDs. This makes containment queries (`@>` , `?` , `@@` , array `&&`) very fast. The trade-off is that GIN indexes are slower to build and update than B-trees, because one row can contribute many keys.

To soften write cost, GIN buffers new entries in a "pending list" (`fastupdate`), flushed in bulk during vacuum or when the list fills. This speeds inserts but can slow reads until the list is merged.

```
CREATE INDEX idx_docs_tags
  ON docs USING gin (tags);

-- "rows whose tags array contains both values"
SELECT * FROM docs WHERE tags @> ARRAY['sql', 'index'];
```

> **GOTCHA**
>
> GIN answers containment, not ordering or range. It cannot serve `ORDER BY` or `<` / `>` on the base column — that is still B-tree territory.

50 GiST and BRIN

INDEXING

Q When do you reach for GiST versus BRIN?

A GiST (Generalized Search Tree) is a balanced-tree framework for data with no natural linear order: geometric shapes, ranges, geometries, nearest-neighbor. The WHY it exists: B-tree assumes a total order, which fails for "do these rectangles overlap?" GiST supports overlap (`&&`), containment, and distance-ordered KNN searches (`ORDER BY geom <-> point`).

BRIN (Block Range Index) is the opposite philosophy: tiny and lossy. It stores only a summary (min/max) per range of table blocks. The WHY it is powerful: on a huge table whose values are physically correlated with position on disk — like an append-only `created_at` — BRIN can prune enormous block ranges while occupying kilobytes, not gigabytes.

Rule of thumb: GiST for spatial/range/similarity where you need precision; BRIN for very large, naturally-ordered tables where a small, approximate index that skips irrelevant blocks is enough.

```
-- GiST for range overlap
CREATE INDEX ON reservations USING gist (during);
-- BRIN for a huge time-ordered table
CREATE INDEX ON events USING brin (created_at);
```

> **GOTCHA**
>
> BRIN collapses if physical ordering does not match value ordering. Random inserts or a heavily updated table destroy the block correlation and make BRIN nearly useless.

51 Indexing JSONB

INDEXING

Q How do you index JSONB columns effectively?

A JSONB supports two GIN strategies. The default operator class indexes every key and value, enabling containment (`@>`), existence (`?`), and path queries. The WHY of a smaller alternative: `jsonb_path_ops` indexes only paths-to-values, producing a smaller, faster index for `@>` containment — at the cost of not supporting the key-existence operators.

If you only ever filter on one scalar field, a plain B-tree expression index on that extracted field is usually best: it is small, ordered, and supports ranges and sorting, which GIN does not.

Choose by query shape: broad ad-hoc containment across many keys favors GIN; a hot lookup on one known field favors a targeted expression B-tree.

```
-- General containment/existence queries
CREATE INDEX idx_doc_data ON docs USING gin (data);
-- Smaller, containment-only
CREATE INDEX ON docs USING gin (data jsonb_path_ops);
-- Targeted B-tree on one field (supports ranges/sort)
CREATE INDEX ON docs ((data->>'status'));
```

> **GOTCHA**
>
> `->` returns `jsonb` ; `->>` returns `text` . Your expression index must use the same operator as the query, or it will not match.

52 Full-text search

INDEXING

Q How do you index full-text search in PostgreSQL?

A Full-text search converts documents into a `tsvector` (normalized, stemmed lexemes with positions) and queries with a `tsquery`, matched by the `@@` operator. The WHY for a GIN index on the tsvector: without it, every row's document is re-parsed and matched at query time; GIN inverts lexeme to rows so a search is a lookup, not a table scan.

Best practice is to store or generate the tsvector as a column — often a `GENERATED ALWAYS AS ... STORED` column — and put a GIN index on it. Use `to_tsvector('english', body)` so a consistent configuration (dictionary, stemming) is applied at both index and query time.

GIN is preferred for static text because lookups are fast; GiST tsvector indexes are lossy and better for frequently updated data. Combine with `ts_rank` for relevance ordering.

```
ALTER TABLE docs ADD COLUMN tsv tsvector
  GENERATED ALWAYS AS (to_tsvector('english', body)) STORED;
CREATE INDEX idx_docs_tsv ON docs USING gin (tsv);

SELECT id FROM docs WHERE tsv @@ to_tsquery('english', 'index & scan');
```

> **GOTCHA**
>
> The text-search configuration must match at write and read time. Indexing with `'english'` but querying with `'simple'` produces different lexemes and silently misses matches.

53 Planner statistics

PERFORMANCE

Q How does the planner use statistics, and what keeps them fresh?

A PostgreSQL's planner is cost-based: it estimates how many rows each operation produces and picks the cheapest plan. Those estimates come from statistics in `pg_statistic` (readable via `pg_stats`): per-column null fraction, number of distinct values, most-common values with frequencies, and a histogram. The WHY it matters: a wrong row estimate cascades — the planner may pick a nested loop expecting 3 rows but actually process 3 million.

`ANALYZE` (run manually or by autovacuum) samples the table and refreshes these stats. After a bulk load or a large data-shape change, stats can be stale and the planner blind. Running `ANALYZE` is the first fix when a plan is inexplicably bad.

For correlated columns the planner assumes independence and can badly underestimate. Extended statistics (`CREATE STATISTICS`) capture functional dependencies and multi-column distinct counts to correct that.

```
ANALYZE orders;

-- Teach the planner that city depends on zip:
CREATE STATISTICS s_orders (dependencies)
  ON zip, city FROM orders;
```

> **GOTCHA**
>
> Increase `default_statistics_target` (or per-column via `ALTER TABLE ... SET STATISTICS`) for skewed columns; the default sample of 100 buckets can miss heavy hitters and produce poor estimates.

54 VACUUM and autovacuum

MAINTENANCE

Q Why do VACUUM and autovacuum matter?

A PostgreSQL uses MVCC: an `UPDATE` or `DELETE` does not overwrite a row; it marks the old version dead and (for updates) writes a new one. The WHY of VACUUM: those dead tuples accumulate and must be reclaimed, or the table grows without bound and scans wade through garbage. VACUUM marks dead tuple space reusable and updates the visibility and free-space maps.

VACUUM also prevents **transaction ID wraparound**. XIDs are 32-bit and cyclic; if the oldest un-frozen XID gets too old, PostgreSQL will refuse writes to protect data. VACUUM freezes old tuples to advance the safe horizon — this is not optional maintenance, it is survival.

Autovacuum automates this, firing when dead tuples exceed a threshold (scale factor times table size). It also triggers autoanalyze to refresh statistics. On write-heavy tables the defaults are often too lazy; tune per-table thresholds so vacuum keeps pace.

```
-- Make autovacuum more aggressive on a hot table:
ALTER TABLE orders SET (
  autovacuum_vacuum_scale_factor = 0.02,
  autovacuum_vacuum_cost_limit   = 2000
);
```

> **GOTCHA**
>
> Plain `VACUUM` does not shrink the file or return space to the OS — it only makes space reusable internally. Only `VACUUM FULL` (which takes an ACCESS EXCLUSIVE lock and rewrites the table) reclaims disk.

55 Table and index bloat

MAINTENANCE

Q What is bloat and how do you deal with it?

A Bloat is space occupied by dead tuples and unused free space inside a table or index — physical size far larger than the live data warrants. The WHY it hurts: bloated relations mean more pages to read, worse cache hit rates, and slower scans even though the live row count is unchanged.

Bloat grows when dead tuples are produced faster than autovacuum reclaims them, or from long-running transactions that hold back the vacuum horizon so old versions cannot be removed. Indexes bloat too, especially under heavy update churn.

To remove existing bloat you must rewrite the object: `VACUUM FULL` or `CLUSTER` (both take heavy locks), or `pg_repack` (online, minimal locking). For index bloat, `REINDEX INDEX CONCURRENTLY` rebuilds without blocking writes.

```
-- Rebuild a bloated index without blocking writes (PG12+):
REINDEX INDEX CONCURRENTLY idx_orders_cust;

-- Watch out for old transactions that pin the vacuum horizon:
SELECT pid, age(backend_xid), query
FROM pg_stat_activity WHERE state <> 'idle';
```

> **GOTCHA**
>
> A single forgotten `idle in transaction` session can block vacuum across the whole database, silently accumulating bloat everywhere. Set `idle_in_transaction_session_timeout`.

56 work_mem and spilling

PERFORMANCE

Q What is work_mem, and what happens when sorts or hashes spill?

A `work_mem` is the memory budget for a single sort, hash, or similar operation within a query. The WHY it is pivotal: if a sort or hash join fits within `work_mem` it runs in RAM (fast); if it exceeds the budget it **spills to disk** as temporary files (much slower). The plan reveals this as `Sort Method: external merge Disk: NNNkB` or extra hash batches.

The subtlety is that `work_mem` is per operation, per node, per connection — not a global cap. A complex query with several sorts and hashes, run by many concurrent connections, can multiply this many times over. Setting it very high globally risks OOM.

The practical approach is a modest global default and targeted per-session bumps for known heavy queries (`SET LOCAL work_mem = '256MB'`). Watch `log_temp_files` to catch spills.

```
EXPLAIN ANALYZE SELECT * FROM big ORDER BY val;
Sort  (cost=...)
  Sort Method: external merge  Disk: 24560kB
-- Raising work_mem lets it become: Sort Method: quicksort  Memory
```

> **GOTCHA**
>
> `hash_mem_multiplier` gives hash-based nodes extra headroom beyond `work_mem` . Tuning only `work_mem` while ignoring it can still leave hash joins spilling.

57 Connection pooling

PERFORMANCE

Q Why is connection pooling needed, and where does PgBouncer fit?

A Each PostgreSQL connection is a separate OS process with its own memory and backend overhead. The WHY of pooling: PostgreSQL does not scale gracefully to thousands of connections — context switching, lock contention, and per-backend memory make many mostly-idle connections expensive. A pool lets a small number of real backends serve a large number of application clients.

PgBouncer is a lightweight external pooler that multiplexes client connections onto a small set of server connections. In `transaction` pooling mode a server connection is assigned only for the duration of a transaction, then returned — this yields the highest reuse and is the common choice for web apps.

Fewer, reused connections mean each avoids the cost of a fresh backend startup and keeps total backend count within what the server handles well. This is often the single biggest scalability fix for a busy app.

```
; pgbouncer.ini
[databases]
app = host=127.0.0.1 dbname=app
[pgbouncer]
pool_mode = transaction
max_client_conn = 5000
default_pool_size = 20
```

> **GOTCHA**
>
> Transaction pooling breaks session-level features: `SET`, session-scoped `PREPARE`, advisory locks, and `LISTEN/NOTIFY` may not survive across statements because you are not guaranteed the same backend. Use session pooling if you need them.

58 Diagnosing slow queries

PERFORMANCE

Q How do you find and diagnose slow queries in production?

A Start with `pg_stat_statements`, an extension that aggregates normalized query statistics: total and mean execution time, call counts, rows, and buffer usage. The WHY of aggregation: a query that takes 20ms but runs a million times an hour hurts more than a nightly 30-second report, and only aggregate totals surface that.

Sort by `total_exec_time` to find where the database actually spends its time, or by `mean_exec_time` for individually slow statements. Because parameters are stripped, all executions of the same shape roll up into one row.

To capture the actual plan of slow statements without manual reproduction, enable `auto_explain` and set `auto_explain.log_min_duration`; it logs the plan of any query exceeding the threshold, optionally with ANALYZE and BUFFERS.

```
SELECT query, calls, round(total_exec_time) AS total_ms,
       round(mean_exec_time, 2) AS mean_ms
FROM pg_stat_statements
ORDER BY total_exec_time DESC LIMIT 10;
```

> **GOTCHA**
>
> `auto_explain` with `log_analyze = on` adds real per-query timing overhead. Enable it selectively; leaving ANALYZE on for every query can measurably slow a busy server.

59 The N+1 query problem

PERFORMANCE

Q What is the N+1 query problem and how do you fix it?

A N+1 is when code runs one query to fetch N parent rows, then one additional query per parent to fetch its children — 1 + N round trips. The WHY it is so costly: even if each query is fast, network and per-statement latency dominate. A hundred parents becomes 101 round trips, and the total time is bounded by round-trip count, not by the work done.

It usually hides behind ORM lazy loading: iterating a collection quietly emits a query per element. The database is not slow — the access pattern is.

The fix is to fetch related data in one round trip: a `JOIN`, or fetching children for all parents at once with `WHERE parent_id = ANY($1)`, or the ORM's eager-loading/batch mechanism. One query returning everything beats N tiny ones.

```
-- Instead of one query per order:
SELECT * FROM items WHERE order_id = $1;   -- run N times

-- Fetch all children in a single round trip:
SELECT * FROM items WHERE order_id = ANY($1);
-- or JSON-aggregate children per parent in one query
```

> **GOTCHA**
>
> Over-correcting into one giant multi-join can explode row counts (a Cartesian product across two child collections). Sometimes two batched queries beat one enormous join.

60 When not to add an index

INDEXING

Q When should you NOT add an index, and what is over-indexing?

A Indexes are not free, so skip them when they will not pay off. The WHY: every index taxes writes, consumes storage, and competes for cache. If it is rarely used or barely selective, that ongoing cost buys almost nothing.

Avoid indexing low-cardinality columns the planner will ignore (a boolean that is mostly one value — a partial index is better). Avoid indexing small tables, where a sequential scan is already cheapest. Avoid redundant indexes: an index on `(a)` is subsumed by `(a, b)` as a left prefix, so the standalone `(a)` may be dead weight. And avoid write-hot tables where index maintenance dominates.

Over-indexing is the accumulation of many overlapping or unused indexes — often from cargo-culting "index every column." It silently slows writes and inflates storage. Audit with `pg_stat_user_indexes` and drop indexes with zero scans.

```
-- Find unused indexes (never scanned, not a constraint):
SELECT relname, indexrelname, idx_scan,
       pg_size_pretty(pg_relation_size(indexrelid)) AS size
FROM pg_stat_user_indexes
WHERE idx_scan = 0
ORDER BY pg_relation_size(indexrelid) DESC;
```

> **GOTCHA**
>
> Before dropping, confirm the index does not back a constraint (primary key, unique, or exclusion) and is not used only by rare-but-critical jobs. Check across a full business cycle, since `idx_scan` resets on stats reset.

61 ACID Properties

TRANSACTIONS

Q What are the ACID properties, and how does PostgreSQL guarantee them?

A ACID stands for Atomicity, Consistency, Isolation, and Durability — the four guarantees that make a transaction a reliable unit of work. The **why** matters: without them, a crash or a concurrent writer could leave your data in a half-applied, contradictory state. PostgreSQL is a fully ACID-compliant relational database, and each property is backed by a concrete mechanism.

Atomicity means all statements in a transaction succeed or none do; PostgreSQL achieves this by only making changes visible on `COMMIT` and discarding them on `ROLLBACK`. **Consistency** means the database moves from one valid state to another, enforced by constraints (primary keys, foreign keys, checks, triggers). **Isolation** means concurrent transactions don't see each other's uncommitted work, implemented through MVCC and snapshots. **Durability** means a committed transaction survives a crash, guaranteed by the write-ahead log (WAL) being flushed to disk before commit returns.

The key insight is that these are not abstract ideals but tunable, observable behaviors. Durability, for instance, depends on `synchronous_commit`; isolation depends on the isolation level you request. Understanding which mechanism backs which property lets you reason about failures precisely.

```
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
-- Atomicity: both updates commit together, or neither does
COMMIT;
```

GOTCHA

Setting `synchronous_commit = off` trades a small durability window (a few hundred ms of committed transactions can be lost on crash) for throughput. The database stays consistent and never corrupts — you only risk losing the very latest commits.

62 BEGIN, COMMIT, ROLLBACK

TRANSACTIONS

Q How do you control transaction boundaries, and what happens to autocommit?

A A transaction is explicitly opened with `BEGIN` (or the SQL-standard `START TRANSACTION`), finalized with `COMMIT` to persist changes, or discarded with `ROLLBACK`. The **why** is that grouping statements gives you all-or-nothing semantics: intermediate states never become visible to other sessions and are undone on failure.

Crucially, PostgreSQL runs in *autocommit* mode by default. Every standalone statement you send is implicitly wrapped in its own transaction that commits immediately if it succeeds. So `BEGIN` is what turns that off, letting you batch multiple statements into one atomic unit. Once inside a transaction block, if any statement raises an error, the entire transaction enters an **aborted** state and every subsequent command is rejected until you `ROLLBACK`.

That aborted-state behavior surprises people coming from MySQL: PostgreSQL will not let you "continue past" a failed statement in a transaction unless you use savepoints. This is a deliberate safety design — it prevents you from accidentally committing a partial result after a constraint violation.

```
BEGIN;
INSERT INTO orders(id, total) VALUES (1, 50);
INSERT INTO orders(id, total) VALUES (1, 99);  -- duplicate key error
-- transaction is now aborted; only ROLLBACK (or COMMIT-as-rollback)
accepted
ROLLBACK;
```

> **GOTCHA**
>
> Issuing `COMMIT` on an already-aborted transaction does *not* commit anything — PostgreSQL silently rolls it back and returns `ROLLBACK`. Always check the command tag if you rely on the result.

63 The Four Isolation Levels

ISOLATION

Q What isolation levels does PostgreSQL support, and what is the default?

A The SQL standard defines four isolation levels — Read Uncommitted, Read Committed, Repeatable Read, and Serializable — and PostgreSQL's default is **Read Committed**. The **why** of isolation levels is that they trade off concurrency against the anomalies a transaction may observe: stricter levels prevent more anomalies but cost more coordination.

PostgreSQL is unusual in that it implements only three distinct behaviors. Read Uncommitted is treated identically to Read Committed — because of MVCC, PostgreSQL simply never exposes uncommitted (dirty) data, so it cannot actually be "less strict." Repeatable Read is implemented as full snapshot isolation, which is stronger than the standard requires. Serializable adds Serializable Snapshot Isolation (SSI) on top, guaranteeing that any set of concurrent serializable transactions produces a result equivalent to some serial ordering.

You set the level per transaction with `SET TRANSACTION ISOLATION LEVEL`, and it must be done before the first query. Most applications correctly stay on Read Committed; you escalate only when you have read-modify-write logic that could be corrupted by concurrent changes.

```
BEGIN ISOLATION LEVEL REPEATABLE READ;
SELECT sum(balance) FROM accounts;
-- entire transaction sees one consistent snapshot
COMMIT;
```

> **GOTCHA**
>
> Requesting `READ UNCOMMITTED` in PostgreSQL is not an error and does not enable dirty reads — you silently get Read Committed. Don't assume portability of this behavior across engines.

64 Read Phenomena

ISOLATION

Q Explain dirty reads, non-repeatable reads, and phantom reads.

A These are the three classic concurrency anomalies the isolation levels are designed to prevent. Understanding them tells you **why** each level exists. A **dirty read** occurs when a transaction reads data written by another transaction that has not yet committed — if that writer rolls back, you acted on data that never existed.

A **non-repeatable read** occurs when you read the same row twice in one transaction and get different values, because a concurrent transaction committed an `UPDATE` or `DELETE` between your two reads. A **phantom read** is similar but affects sets: you run the same query twice with a `WHERE` clause and new rows *appear* (or vanish) because another transaction committed an `INSERT` or `DELETE` matching your predicate.

In PostgreSQL: dirty reads are impossible at every level thanks to MVCC. Read Committed permits non-repeatable and phantom reads. Repeatable Read (snapshot isolation) prevents non-repeatable reads and, unlike the bare SQL standard, also prevents phantom reads. Serializable additionally prevents subtler serialization anomalies such as write skew, which none of the simpler read phenomena capture.

```
-- Non-repeatable read under READ COMMITTED:
SELECT balance FROM accounts WHERE id = 1;  -- returns 500
-- another session commits UPDATE ... SET balance = 300
SELECT balance FROM accounts WHERE id = 1;  -- now returns 300
```

> **GOTCHA**
>
> Repeatable Read in PostgreSQL blocks phantoms too, but it does *not* prevent **write skew** — two transactions each reading a consistent snapshot and writing disjoint rows based on it. Only Serializable catches that.

65 MVCC: xmin and xmax

MVCC

Q How does PostgreSQL implement MVCC with tuple versions?

A MVCC (Multi-Version Concurrency Control) is PostgreSQL's core concurrency mechanism, and the **why** is elegant: instead of locking rows for reads, the database keeps multiple physical versions of each row and shows each transaction the version appropriate to its snapshot. Every row version (called a *tuple*) carries two hidden system columns, `xmin` and `xmax`.

`xmin` is the transaction ID (XID) that *created* the tuple; `xmax` is the XID that *deleted or superseded* it (0 if still live). An `UPDATE` is not an in-place edit — it marks the old tuple's `xmax` and inserts a brand-new tuple with a fresh `xmin`. A `DELETE` just stamps `xmax`. When a transaction reads a page, it checks each tuple's `xmin` / `xmax` against its snapshot and the commit status to decide visibility.

This append-style design is why PostgreSQL never overwrites live data during normal operation, and why it produces dead tuples that must later be reclaimed. You can literally see these columns by selecting them explicitly.

```
SELECT xmin, xmax, id, balance FROM accounts WHERE id = 1;
-- xmin = creating XID, xmax = 0 means the row is still live
```

> **GOTCHA**
>
> Because `UPDATE` writes a whole new tuple, updating even one small column rewrites the entire row and every non-HOT index entry. This is why wide, frequently-updated tables bloat quickly — consider HOT updates by leaving fillfactor headroom and not indexing hot columns.

66 Readers Don't Block Writers

MVCC

Q Why does MVCC mean readers don't block writers and vice versa?

A This is the headline benefit of MVCC. Because every writer creates a *new* version of a row rather than mutating the existing one in place, a reader can continue to see the old committed version while a writer produces a new one. The **why**: visibility is determined by snapshot and XID comparison, not by a shared read lock, so reads need no lock at all.

Concretely, a `SELECT` never has to wait for an `UPDATE` on the same row, and an `UPDATE` never waits for concurrent `SELECT`s. This dramatically improves concurrency for read-heavy workloads compared to lock-based systems where readers acquire shared locks that block writers. Only writer-versus-writer conflicts on the *same row* require waiting, because two transactions cannot both own the newest version.

The trade-off is the cost of maintaining old versions (dead tuples) and the need for VACUUM to clean them. But for the vast majority of OLTP workloads, non-blocking reads are a decisive win, and it's the reason PostgreSQL scales gracefully under mixed read/write load.

```
-- Session A (long report) does not block Session B (update):
-- A: SELECT sum(balance) FROM accounts;  -- runs on its snapshot
-- B: UPDATE accounts SET balance = balance + 1;  -- proceeds freely
```

> **GOTCHA**
>
> Two writers *do* block each other on the same row: the second `UPDATE` waits for the first to commit or roll back, then re-evaluates. Under Read Committed it re-reads the updated row; under Repeatable Read it aborts with a serialization error.

67 Dead Tuples and VACUUM

MVCC

Q What are dead tuples, and how does VACUUM relate to them?

A A *dead tuple* is a row version that is no longer visible to any current or future transaction — the leftover of an `UPDATE` or `DELETE`. The **why** they exist is a direct consequence of MVCC: old versions must linger until no running snapshot could still need them. Once every transaction that might see them has ended, they become garbage occupying disk pages.

`VACUUM` is the process that reclaims the space dead tuples occupy, marking it reusable within the table (plain VACUUM) or returning it to the OS (VACUUM FULL, which rewrites the whole table and takes an exclusive lock). Autovacuum runs this automatically based on how many rows have changed. Without it, tables and indexes *bloat* — they grow far larger than the live data, slowing scans and wasting cache.

VACUUM does more than reclaim space: it also updates the visibility map (enabling index-only scans), and it *freezes* old tuples to prevent transaction ID wraparound. So VACUUM is not merely cleanup — it is essential to correctness and performance both.

```
SELECT relname, n_dead_tup, n_live_tup, last_autovacuum
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC;
-- high n_dead_tup relative to n_live_tup signals bloat
```

> **GOTCHA**
>
> A single long-running transaction (even an idle one holding a snapshot) prevents VACUUM from removing dead tuples newer than its snapshot — *anywhere* in the database. This is the most common cause of runaway bloat.

68 Repeatable Read (Snapshot Isolation)

ISOLATION

Q How does Repeatable Read work in PostgreSQL, and how is it stronger than the standard?

A Under Repeatable Read, PostgreSQL takes a single snapshot at the moment of the transaction's *first* statement and every subsequent read sees the database exactly as of that instant. The **why**: a stable snapshot guarantees that anything you read once will read identically later, eliminating non-repeatable reads entirely.

Because the snapshot is a consistent point-in-time view, PostgreSQL's implementation also blocks phantom reads — new committed rows from other transactions simply aren't in your snapshot. This is *snapshot isolation*, and it is stronger than the SQL standard's minimum for Repeatable Read (which technically still allows phantoms). This is why PostgreSQL's Repeatable Read is genuinely useful for consistent reporting: a multi-query report sees one coherent world.

The cost appears on writes. If your Repeatable Read transaction tries to update a row that another transaction modified and committed after your snapshot, PostgreSQL cannot silently proceed — it raises a serialization failure (`40001`) and you must retry the whole transaction.

```
BEGIN ISOLATION LEVEL REPEATABLE READ;
SELECT count(*) FROM events;   -- snapshot fixed here
-- ... other sessions insert events ...
SELECT count(*) FROM events;   -- identical count, no phantoms
COMMIT;
```

> **GOTCHA**
>
> The snapshot is taken at the first query, not at `BEGIN` . If you `BEGIN ISOLATION LEVEL REPEATABLE READ` and then sit idle before your first `SELECT` , the snapshot reflects that later moment, not the `BEGIN` time.

69 Serializable Isolation (SSI)

ISOLATION

Q How does Serializable isolation work, and what is SSI?

A Serializable is the strictest level: it guarantees that the outcome of concurrently running serializable transactions is identical to *some* serial (one-at-a-time) execution of them. The **why** is that snapshot isolation alone permits anomalies like write skew, where two transactions each read overlapping data and write disjoint rows in a way no serial order could produce.

PostgreSQL implements this with **Serializable Snapshot Isolation (SSI)**, an optimistic algorithm layered on snapshot isolation. It does not add read locks; instead it tracks *predicate* dependencies (via SIRead "shared" locks that only monitor, never block) and detects dangerous patterns of read-write conflicts that could form a non-serializable cycle. When it finds one, it aborts one of the transactions with a serialization failure rather than committing an inconsistent result.

The beauty of SSI is that it preserves MVCC's non-blocking reads — performance under low contention is close to Repeatable Read — while providing true serializability. The cost is more aborts under contention and some memory for tracking predicate locks.

```
BEGIN ISOLATION LEVEL SERIALIZABLE;
SELECT sum(hours) FROM shifts WHERE on_call;
-- decide whether this worker can go off-call, then:
UPDATE shifts SET on_call = false WHERE worker_id = 7;
COMMIT;  -- may raise 40001 if a concurrent txn broke serializability
```

> **GOTCHA**
>
> For SSI to protect you, *all* participating transactions must run at Serializable. A single concurrent Read Committed transaction touching the same data can introduce the anomaly SSI was meant to prevent, because SSI only tracks conflicts among serializable transactions.

70 Handling serialization_failure

CONCURRENCY

Q How should application code handle serialization failures?

A When you run at Repeatable Read or Serializable, PostgreSQL can abort a transaction with SQLSTATE `40001` (`serialization_failure`) or `40P01` (`deadlock_detected`). The **why**: these are not bugs or data errors — they are the database telling you it could not safely serialize your work, and the correct, expected response is simply to *retry the entire transaction*.

The retry must re-run all the logic from `BEGIN`, because the transaction's snapshot is gone and any values you read are stale. You cannot just re-issue the failing statement. A robust implementation wraps the transaction in a loop with a bounded number of attempts and ideally a small randomized (exponential) backoff to avoid thundering-herd contention where the same transactions keep colliding.

This retry pattern is what makes serializable isolation practical: you write straightforward business logic as if you were alone, and let the database detect conflicts, trading a rare retry for not having to hand-craft locking. The application, not the database, owns the retry.

```
# pseudocode
for attempt in range(MAX_RETRIES):
    try:
        run_transaction()   # BEGIN ... COMMIT
        break
    except SerializationFailure:   # SQLSTATE 40001 / 40P01
        sleep(random() * 2**attempt * 0.01)
```

> **GOTCHA**
>
> Any side effect inside the transaction that isn't rolled back — sending an email, calling an external API — will fire again on retry. Keep external side effects *out* of retryable transactions, or make them idempotent.

71 Row-Level Locks: FOR UPDATE / FOR SHARE

LOCKING

Q What do SELECT ... FOR UPDATE and FOR SHARE do?

A These are explicit row-level locking clauses on a `SELECT`, used to reserve rows you intend to modify. The **why**: MVCC lets reads proceed without locks, but sometimes you read a row *in order to* update it and need to stop other transactions from changing it in between — a read-modify-write. `FOR UPDATE` takes an exclusive row lock so no other transaction can update, delete, or also `FOR UPDATE`-lock those rows until you commit.

`FOR SHARE` takes a weaker shared lock: multiple transactions can hold it simultaneously (all "just reading, don't change this"), but any writer must wait. There are also `FOR NO KEY UPDATE` and `FOR KEY SHARE`, weaker variants that conflict less and matter for foreign-key maintenance concurrency. Locks are held until the end of the transaction, never released early.

The classic use is safely decrementing inventory or moving money: lock the row, read the current value, compute, update, commit. Because the lock blocks concurrent writers, you avoid the lost-update anomaly without escalating to Serializable isolation.

```
BEGIN;
SELECT stock FROM products WHERE id = 42 FOR UPDATE;
-- other writers to row 42 now wait here
UPDATE products SET stock = stock - 1 WHERE id = 42;
COMMIT;
```

> **GOTCHA**
>
> Add `NOWAIT` to fail instantly instead of blocking, or `SKIP LOCKED` to ignore already-locked rows. Also beware: `FOR UPDATE` cannot be used with aggregates, `GROUP BY`, or `DISTINCT`.

72 Table-Level Locks

LOCKING

Q What are table-level lock modes and when does LOCK matter?

A PostgreSQL has eight table-level lock modes, from `ACCESS SHARE` (weakest, taken by `SELECT`) up to `ACCESS EXCLUSIVE` (strongest, taken by `DROP`, `TRUNCATE`, and many `ALTER TABLE` forms). The **why**: even though row-level MVCC handles most concurrency, DDL and maintenance operations need coarser coordination, and each command acquires exactly the lock strength it needs so that compatible operations can still run in parallel.

The rule that matters is the conflict matrix: `ACCESS SHARE` conflicts only with `ACCESS EXCLUSIVE`. So ordinary reads and writes never block each other at the table level, but an `ACCESS EXCLUSIVE` operation blocks everything, including plain `SELECT`s. You can request a lock explicitly with the `LOCK TABLE` command, though most locks are acquired automatically by the statements you run.

Understanding this is essential for safe schema migrations. An `ALTER TABLE ... ADD COLUMN` with a volatile default, or adding a constraint that scans the table, can hold `ACCESS EXCLUSIVE` for the whole scan and freeze production traffic.

```
BEGIN;
LOCK TABLE reports IN SHARE MODE;  -- blocks writers, allows readers
INSERT INTO report_summary SELECT ... FROM reports;
COMMIT;
```

> **GOTCHA**
>
> Set `lock_timeout` before running DDL on a busy table. Otherwise an `ALTER` waiting behind a long `SELECT` will queue its `ACCESS EXCLUSIVE` request, and every new query then queues behind *it* — a full stall.

73 Deadlocks

LOCKING

Q How do deadlocks happen and how does PostgreSQL resolve them?

A A deadlock is a cycle of waiting: transaction A holds a lock B needs, while B holds a lock A needs, so neither can proceed. The **why** it arises is almost always *inconsistent lock ordering* — two transactions acquire the same set of locks in different orders. The textbook case is A locking row 1 then row 2 while B locks row 2 then row 1.

PostgreSQL cannot prevent deadlocks (that would require global lock coordination), so instead it *detects* them. A background check runs after `deadlock_timeout` (default one second) whenever a transaction has been waiting; it builds a wait-for graph and looks for a cycle. On finding one, it chooses a *victim* and aborts it with SQLSTATE `40P01` (`deadlock_detected`), releasing its locks so the other transaction proceeds.

The victim's application should catch that error and retry, exactly like a serialization failure. The durable fix, though, is prevention: always acquire locks in a consistent, deterministic order (e.g., always update accounts in ascending id order) across all code paths.

```
-- Session A                    -- Session B
UPDATE t SET x=1 WHERE id=1;    UPDATE t SET x=1 WHERE id=2;
UPDATE t SET x=1 WHERE id=2;    UPDATE t SET x=1 WHERE id=1;
-- one session is aborted with deadlock_detected (40P01)
```

> **GOTCHA**
>
> Lowering `deadlock_timeout` makes detection faster but the check is expensive; it's a timeout, not a poll interval. Frequent deadlocks mean your code has an ordering bug — fix the ordering rather than tuning the timeout.

74 Optimistic vs Pessimistic Locking

CONCURRENCY

Q What's the difference between optimistic and pessimistic locking?

A These are two strategies for handling concurrent updates to the same data. **Pessimistic** locking assumes conflicts are likely and prevents them up front by acquiring a lock before modifying — e.g. `SELECT ... FOR UPDATE`. Other writers wait. The **why** to choose it: when contention is high, blocking is cheaper than repeatedly doing work that gets thrown away.

Optimistic locking assumes conflicts are rare and takes no lock. Instead it reads a version marker (a `version` column or `updated_at`), does its work, and on write checks that the marker hasn't changed. If it has, someone else won the race and the update affects zero rows — the application detects that and retries. No locks are held during the think-time, which maximizes concurrency for read-heavy, low-conflict workloads.

In PostgreSQL, Serializable/Repeatable Read isolation is essentially database-managed optimistic concurrency (it aborts on conflict), while `FOR UPDATE` is pessimistic. Choose pessimistic for hot rows and short transactions; optimistic for rarely-contended rows or long user "edit form" sessions where holding a lock would be untenable.

```
-- Optimistic: succeeds only if version is unchanged
UPDATE docs SET body = 'new', version = version + 1
WHERE id = 10 AND version = 3;
-- 0 rows updated => someone else changed it; reload and retry
```

> **GOTCHA**
>
> Optimistic locking only works if *every* writer honors the version check. One code path that updates the row without bumping/checking `version` silently defeats the whole scheme and reintroduces lost updates.

75 Advisory Locks

LOCKING

Q What are advisory locks and when would you use them?

A Advisory locks are application-defined locks identified by an integer (or two 32-bit integers) that have *no* connection to any table or row. The **why**: sometimes you need mutual exclusion over a concept that isn't a database row — "only one worker may run this cron job," "serialize processing for customer 500" — and advisory locks give you a fast, in-database mutex without creating a lock table.

They come in two lifetimes. Session-level locks (`pg_advisory_lock`) persist until explicitly unlocked or the session ends. Transaction-level locks (`pg_advisory_xact_lock`) auto-release at transaction end, which is usually safer because you can't leak them. Each has a non-blocking `pg_try_advisory_lock` variant that returns a boolean immediately instead of waiting.

Because PostgreSQL does not interpret the integer, it's on *you* to establish a convention mapping meanings to key values consistently across your codebase, or two unrelated features will collide on the same key.

```
-- Only one instance runs the nightly job:
SELECT pg_try_advisory_lock(42);   -- true = got it, false = someone else
has it
-- ... do work ...
SELECT pg_advisory_unlock(42);
```

> **GOTCHA**
>
> Session-level advisory locks survive across transactions and are *not* released by `ROLLBACK` . If your connection is pooled, a leaked session lock can be inherited by the next request on that connection — prefer the `_xact_` variants.

76 Savepoints and Nested Transactions

TRANSACTIONS

Q How do savepoints work, and does PostgreSQL support nested transactions?

A PostgreSQL does not support truly nested top-level transactions, but it offers *savepoints*, which provide subtransaction semantics inside one transaction. The **why**: you sometimes want to attempt something risky and, if it fails, undo only that part while keeping the rest of the transaction alive — recall that a raw error otherwise aborts the whole transaction.

You establish a marker with `SAVEPOINT name`, and can later `ROLLBACK TO SAVEPOINT name` to discard everything done since the savepoint (including recovering from an error state) without discarding earlier work. `RELEASE SAVEPOINT` forgets the marker, keeping its changes. This is exactly how a driver's "nested transaction" or an ORM's nested `with transaction` block is implemented under the hood.

Savepoints are what let you write a loop that tries each row and skips the ones that violate a constraint, without aborting the batch. But they aren't free: each subtransaction consumes a subtransaction XID.

```
BEGIN;
INSERT INTO t VALUES (1);
SAVEPOINT sp1;
INSERT INTO t VALUES (1);   -- duplicate key: subtransaction aborts
ROLLBACK TO SAVEPOINT sp1; -- recover; row (1) from before is kept
INSERT INTO t VALUES (2);
COMMIT;
```

> **GOTCHA**
>
> Heavy savepoint use (e.g. a savepoint per row in a huge loop) can blow past the 64-entry subtransaction cache and cause `SubtransSLRU` contention, degrading performance server-wide. Batch differently if you're creating thousands of subtransactions.

77 FOR UPDATE SKIP LOCKED

LOCKING

Q How does SKIP LOCKED enable a job-queue pattern?

A Adding `SKIP LOCKED` to `SELECT ... FOR UPDATE` tells PostgreSQL to silently skip over any rows currently locked by another transaction instead of waiting for them. The **why** this is powerful: it lets many workers pull disjoint sets of rows from the same queue table concurrently, with zero contention and no external coordination, because each worker grabs only rows nobody else has locked.

The pattern is a canonical work queue: each worker runs a `SELECT` for the next pending job with `FOR UPDATE SKIP LOCKED LIMIT 1`, processes it, marks it done, and commits — releasing the lock. Two workers running simultaneously will never grab the same job, and neither blocks the other. This turns a plain table into a reliable, transactional job queue without a dedicated message broker.

Contrast this with plain `FOR UPDATE` (workers serialize, all waiting for the first row) or `NOWAIT` (workers error out on contention). `SKIP LOCKED` is the sweet spot for competing consumers.

```
BEGIN;
SELECT id, payload FROM jobs
WHERE status = 'pending'
ORDER BY created_at
FOR UPDATE SKIP LOCKED
LIMIT 1;
-- process, then:
UPDATE jobs SET status = 'done' WHERE id = :id;
COMMIT;
```

GOTCHA

`SKIP LOCKED` gives at-least-once semantics only if you commit the status change in the *same* transaction that locked the row. If the worker crashes mid-process, the lock releases on disconnect and the job becomes claimable again — make processing idempotent.

78 Long-Running Transactions

CONCURRENCY

Q Why are long-running transactions dangerous in PostgreSQL?

A A transaction that stays open for a long time — even an idle one that just holds a snapshot — is one of the most damaging things you can do to a PostgreSQL database. The **why** traces straight back to MVCC: as long as an old transaction's snapshot is alive, VACUUM cannot remove *any* dead tuple that might still be visible to it, across the whole cluster.

The consequence is unbounded bloat. Dead tuples from busy, unrelated tables accumulate because the oldest snapshot pins them, so tables and indexes grow, cache hit rates fall, and scans slow down — a slow, systemic degradation. Worse, VACUUM's inability to advance also prevents *freezing*, which pushes the database toward transaction ID wraparound (the next topic), an emergency that can force read-only mode.

Long transactions also hold locks longer, blocking DDL and other writers. The lesson: keep transactions short, never leave a transaction open across user think-time or a slow network call, and monitor for old transactions actively.

```
-- Find the oldest running transactions:
SELECT pid, now() - xact_start AS age, state, query
FROM pg_stat_activity
WHERE xact_start IS NOT NULL
ORDER BY age DESC;
```

> **GOTCHA**
>
> The worst offender is `idle in transaction` — a connection that ran `BEGIN`, did one query, and then sat waiting on the app. Set `idle_in_transaction_session_timeout` to have PostgreSQL kill these automatically.

79 Transaction ID Wraparound and Freezing

MVCC

Q What is transaction ID wraparound, and how does freezing prevent it?

A Transaction IDs (XIDs) in PostgreSQL are 32-bit numbers, so there are only about 4 billion of them, and they are compared in a *circular* (modulo) space where roughly 2 billion XIDs are "in the past" and 2 billion are "in the future" relative to any point. The **why** wraparound is dangerous: if a live tuple's `xmin` ever fell more than ~2 billion transactions behind the current XID, it would suddenly appear to be in the *future* and become invisible — silent, catastrophic data loss.

To prevent this, VACUUM *freezes* old tuples: it marks tuples older than a threshold as "frozen," meaning permanently in the past and always visible regardless of XID comparison. Freezing removes those tuples from the wraparound race entirely. Autovacuum triggers an aggressive anti-wraparound vacuum as the oldest unfrozen XID ages, governed by `autovacuum_freeze_max_age`.

If freezing falls too far behind (often because a long transaction or a stuck autovacuum blocked it), PostgreSQL first warns loudly, then refuses new XIDs and goes read-only to protect your data — you must run VACUUM to recover. Modern versions store XIDs as 64-bit in some internal structures, but heap tuple XIDs remain 32-bit, so freezing still matters.

```
-- How close is each database to wraparound trouble?
SELECT datname, age(datfrozenxid) AS xid_age
FROM pg_database
ORDER BY xid_age DESC;
-- approaching 2^31 (~2.1B) means act now
```

> **GOTCHA**
>
> Anything that stalls VACUUM — long transactions, abandoned replication slots, prepared transactions left uncommitted — also stalls freezing and marches you toward wraparound. Monitor `age(datfrozenxid)`; don't wait for the warnings.

80 Idempotency and Safe Retries

TRANSACTIONS

Q How do you write transactional code that is safe to retry?

A Because serialization failures, deadlocks, and network blips all force retries, and because clients may resend a request they never got a response for, your transactional operations should be *idempotent*: running them twice produces the same end state as running them once. The **why**: retries are inevitable, so correctness cannot depend on a transaction executing exactly once.

The most robust technique is an idempotency key — a unique token supplied by the client and stored with a unique constraint. The transaction inserts the key; if the insert violates the unique constraint, the operation already ran and you return the prior result instead of doing the work again. `INSERT ... ON CONFLICT DO NOTHING` makes this a single atomic step. This turns "process payment" from a dangerous double-charge risk into a safe, replayable operation.

Combine this with the earlier retry rules: keep external side effects (emails, third-party API calls) out of the retryable transaction or make them idempotent too, and don't rely on values read before an aborted attempt. Idempotency plus a retry loop is the foundation of reliable transactional systems.

```
INSERT INTO payments (idempotency_key, amount, status)
VALUES ('req-abc-123', 5000, 'charged')
ON CONFLICT (idempotency_key) DO NOTHING
RETURNING id;
-- empty result => this request was already processed; return the existing
row
```

> **GOTCHA**
>
> A `SELECT` -then- `INSERT` "check if it exists first" is *not* idempotent under concurrency — two retries can both pass the check and both insert. Rely on the unique constraint and `ON CONFLICT` to make atomicity the database's job, not the application's.

81 Roles, Users, and Privileges

ADMINISTRATION

Q How do roles, users, and privileges work in PostgreSQL, and how do GRANT/REVOKE apply?

A PostgreSQL has no separate concept of "users" and "groups" internally, only `ROLE`. A role is any entity that can own objects and hold privileges. `CREATE USER` is merely syntactic sugar for `CREATE ROLE ... LOGIN`. This unification is the key insight: a role that has the `LOGIN` attribute can authenticate and connect, while a role without it typically functions as a group you **GRANT membership into**.

Privileges are granted on objects (tables, sequences, schemas, databases, functions) with `GRANT` and removed with `REVOKE`. Because roles can be members of other roles, you build a hierarchy: grant privileges to a group role, then grant that group to individual login roles. Membership can be used automatically or require an explicit `SET ROLE`, depending on the `INHERIT` attribute.

A crucial subtlety is that granting privileges on existing tables does not affect tables created later. To make future objects inherit privileges automatically, use `ALTER DEFAULT PRIVILEGES`. Also note that the object owner and superusers always have full rights regardless of GRANT.

```
CREATE ROLE app_read -- group role, no login;
CREATE ROLE alice LOGIN PASSWORD 'secret';
GRANT USAGE ON SCHEMA public TO app_read;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO app_read;
GRANT app_read TO alice;
ALTER DEFAULT PRIVILEGES IN SCHEMA public
  GRANT SELECT ON TABLES TO app_read;
```

GOTCHA

By default, every role can create objects in the `public` schema on older versions. Since PostgreSQL 15 the default `CREATE` privilege on `public` was revoked from `PUBLIC`, tightening security. Always verify schema-level `USAGE` and `CREATE` grants explicitly.

82 Authentication and pg_hba.conf

ADMINISTRATION

Q How does client authentication work, and what is the role of pg_hba.conf?

A Authentication in PostgreSQL is controlled by `pg_hba.conf` ("host-based authentication"). It is completely separate from privileges: pg_hba decides **whether a connection is allowed and how identity is proven**, while GRANT/REVOKE decide what an authenticated role may do. The file is read top-to-bottom and the **first matching rule wins**, so ordering is critical.

Each line specifies a connection type (`local`, `host`, `hostssl`, `hostnossl`), a database, a role, a client address range (CIDR), and an authentication method. Common methods include `scram-sha-256` (the modern password method, preferred over the older `md5`), `peer` (matches the OS username for local socket connections), `cert` (TLS client certificates), and `reject` / `trust`.

After editing, changes take effect on `SELECT pg_reload_conf()` or `pg_ctl reload`, not requiring a full restart. Because a broad early rule can shadow a narrower later one, put specific rules first.

```
# TYPE  DATABASE  USER   ADDRESS         METHOD
local   all       all                    peer
hostssl app_db    alice  10.0.0.0/24     scram-sha-256
host    all       all    0.0.0.0/0       reject
```

> **GOTCHA**
>
> `trust` means no password at all, anyone who can reach the port connects as any role. Never use it on non-loopback interfaces. Setting a password with `ALTER ROLE` is useless if pg_hba uses `trust` for that connection.

83 Row-Level Security (RLS)

SECURITY

Q What is Row-Level Security and how do you implement multi-tenant isolation with it?

A Row-Level Security lets you attach **policies** to a table so that each role only sees or modifies the rows a boolean expression permits. The **why** is that it pushes access control into the database engine itself, so isolation holds even if application code has a bug, rather than relying on every query to remember a `WHERE tenant_id = ...` clause.

You must both enable RLS on the table with `ALTER TABLE ... ENABLE ROW LEVEL SECURITY` and create at least one `POLICY`. Once enabled, if no policy matches, the default is to deny (return zero rows). Policies can differ by command via `FOR SELECT`, `FOR INSERT`, etc., and use a `USING` clause (which rows are visible) and a `WITH CHECK` clause (which rows may be written).

A common multi-tenant pattern combines RLS with a session variable set per request, read via `current_setting()`. The app sets the current tenant once and every query is automatically scoped.

```
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation ON orders
  USING (tenant_id = current_setting('app.tenant_id')::int)
  WITH CHECK (tenant_id = current_setting('app.tenant_id')::int);

-- per request:
SET app.tenant_id = '42';
```

> **GOTCHA**
>
> Table owners and superusers **bypass RLS by default**. Use `ALTER TABLE ... FORCE ROW LEVEL SECURITY` to apply policies even to the owner, and be aware that `BYPASSRLS` role attribute skips it entirely.

84 Logical vs Physical Backups

BACKUP

Q What is the difference between pg_dump/pg_restore and pg_basebackup?

A These operate at fundamentally different levels. `pg_dump` is a **logical** backup: it produces SQL statements (or an archive) that recreate schema and data. `pg_basebackup` is a **physical** backup: a byte-for-byte copy of the data directory files plus the WAL needed to make them consistent. The distinction drives everything about how you use them.

Logical dumps are portable across major versions and even architectures, let you restore a single table, and are human-inspectable. But they are slow to restore on large databases because everything is re-inserted and indexes rebuilt. Use the custom or directory format (`-Fc` / `-Fd`) so `pg_restore` can do parallel, selective restores.

Physical backups are fast for large clusters and are the foundation for replication and point-in-time recovery, but they copy the entire cluster (all databases), are tied to the same major version and platform, and cannot restore a single table. They also form the "base" for PITR when combined with archived WAL.

```
# logical, parallel-restorable custom format
pg_dump -Fc -f db.dump mydb
pg_restore -d mydb -j 4 db.dump

# physical base backup of the whole cluster
pg_basebackup -D /backup/base -Fp -Xs -P
```

> **GOTCHA**
>
> `pg_dump` takes a consistent snapshot via MVCC without blocking writers, but it is not a point-in-time recovery tool. For continuous recoverability you need physical backups plus WAL archiving, not periodic dumps.

85 WAL and Point-In-Time Recovery

BACKUP

Q What is the Write-Ahead Log and how does Point-In-Time Recovery work?

A The Write-Ahead Log (WAL) is the durability mechanism: before any data page change is written to the heap, the change is first recorded in the WAL and flushed to disk. This is why a crash is survivable, on restart PostgreSQL replays WAL to bring data files to a consistent state. The **write-ahead** ordering is what guarantees the "D" in ACID.

Point-In-Time Recovery leverages this. You take a physical base backup and continuously archive WAL segments to a safe location via `archive_command` (or a tool like pgBackRest/WAL-G). To recover, you restore the base backup and replay archived WAL up to a chosen target, a timestamp, transaction ID, named restore point, or LSN. This lets you rewind to just before an accidental `DROP TABLE`.

Recovery is driven by `restore_command` and target settings in the config, with the presence of a `recovery.signal` file (PostgreSQL 12+) putting the server into archive recovery mode.

```
# postgresql.conf
wal_level = replica
archive_mode = on
archive_command = 'cp %p /archive/%f'

# recovery target for PITR
recovery_target_time = '2026-07-21 09:30:00'
restore_command = 'cp /archive/%f %p'
```

> **GOTCHA**
>
> PITR can only roll **forward** from the base backup, never before it. If your only base backup is newer than the moment you want, you cannot recover to that point. Take base backups regularly and keep enough WAL to bridge the gap.

86 Streaming Replication

REPLICATION

Q How does primary/standby streaming replication work?

A Streaming replication ships the primary's WAL records to one or more standby servers over a network connection in near real time, and each standby replays them to stay a nearly-identical copy. The **why** is that WAL already describes every change, so streaming it is a cheap, byte-level way to keep replicas current without re-running SQL.

A standby is created from a base backup of the primary and then connects using a replication connection (a role with `REPLICATION` privilege) via `primary_conninfo`. Standbys are read-only ("hot standby") and can serve read queries, offloading the primary. Replication can be **asynchronous** (default, low latency but possible data loss on failover) or **synchronous** (the primary waits for the standby to confirm, zero data loss but higher commit latency).

Replication slots (`pg_create_physical_replication_slot`) prevent the primary from recycling WAL a standby still needs, avoiding the standby falling irrecoverably behind.

```
# on standby: postgresql.conf
primary_conninfo = 'host=primary user=repl'
primary_slot_name = 'standby1'

# synchronous config on primary
synchronous_standby_names = 'standby1'
```

> **GOTCHA**
>
> An unused replication slot is dangerous: if a standby disappears, its slot pins WAL forever and can fill the primary's disk. Monitor `pg_replication_slots` and set `max_slot_wal_keep_size` to cap retention.

87 Logical Replication and Use Cases

REPLICATION

Q What is logical replication and when should you use it instead of streaming replication?

A Logical replication replicates **individual row changes** (INSERT/UPDATE/DELETE) decoded from the WAL into a logical stream, rather than shipping raw physical WAL blocks. The key difference is granularity and flexibility: you publish specific tables and subscribers apply the changes, so source and target can differ in version, schema layout, indexes, and even have their own additional tables.

It is built on a publisher/subscriber model: `CREATE PUBLICATION` on the source names the tables, and `CREATE SUBSCRIPTION` on the target pulls and applies them. Because it is logical, the subscriber is a fully writable, independent database, unlike a physical standby which is read-only and a whole-cluster mirror.

Typical use cases: **major-version upgrades with near-zero downtime** (replicate from old to new, then switch), consolidating multiple databases into one, selective replication of a subset of tables, and feeding data into analytics or other systems.

```
-- on publisher
CREATE PUBLICATION sales_pub FOR TABLE orders, customers;

-- on subscriber
CREATE SUBSCRIPTION sales_sub
  CONNECTION 'host=src dbname=app user=repl'
  PUBLICATION sales_pub;
```

> **GOTCHA**
>
> Logical replication does not automatically replicate DDL, schema changes, sequences' current values, or (by default) tables without a primary key or replica identity. Adding a column on the publisher and not the subscriber will break apply.

88 High Availability and Failover

OPERATIONS

Q What are the key concepts behind high availability and automatic failover?

A High availability means the database service survives the loss of a node with minimal downtime. PostgreSQL provides the building blocks (replication, hot standbys) but **not automatic failover out of the box**, that orchestration is the job of external tools. The core idea is having a standby ready to be promoted to primary when the current primary fails.

Promotion turns a read-only standby into a writable primary (`pg_ctl promote` or `SELECT pg_promote()`). The hard problems are **detecting** real failure (avoiding false positives), avoiding **split-brain** (two primaries accepting writes) usually via fencing/STONITH and quorum, and redirecting clients to the new primary through a virtual IP, load balancer, or connection router.

Popular tooling includes Patroni (using a distributed consensus store like etcd for leader election), repmgr, and Pacemaker. Key metrics are RPO (how much data you can lose, driven by sync vs async replication) and RTO (how fast you recover).

```
-- promote a standby to primary
SELECT pg_promote();

-- check who is primary vs standby
SELECT pg_is_in_recovery();  -- true = standby
```

> **GOTCHA**
>
> Split-brain is the classic failure: a network partition makes the standby promote itself while the old primary is still alive and taking writes. A proper HA setup requires fencing plus a quorum so only one node can win, never rely on timeouts alone.

89 Declarative Table Partitioning

PERFORMANCE

Q How does declarative partitioning work, and when do you use range, list, or hash?

A Declarative partitioning (PostgreSQL 10+) splits one logical table into physical child tables based on a partition key, letting the planner skip irrelevant partitions (**partition pruning**) and letting you drop or archive old data instantly by detaching a partition. The **why** is manageability and query performance on very large tables, not a magic speedup for small ones.

You choose the strategy by data shape. `RANGE` suits ordered/continuous keys like dates (partition by month), enabling cheap time-based retention. `LIST` suits discrete categories like region or status. `HASH` distributes rows evenly across a fixed number of partitions when there is no natural range/list key, useful for spreading write load.

Each partition is a real table; you can index them individually and even sub-partition. Constraints and indexes defined on the parent propagate to children in modern versions.

```
CREATE TABLE events (id bigint, ts timestamptz, payload jsonb)
  PARTITION BY RANGE (ts);

CREATE TABLE events_2026_07 PARTITION OF events
  FOR VALUES FROM ('2026-07-01') TO ('2026-08-01');
```

> **GOTCHA**
>
> Pruning only works if the query filters on the partition key. A query on `payload` alone scans every partition. Also, the partition key must be part of any `PRIMARY KEY` or `UNIQUE` constraint on a partitioned table.

90 JSONB Operators and When to Use It

JSON

Q What do the JSONB operators do, and when should you prefer JSONB over regular columns?

A JSONB stores JSON in a decomposed binary form that is fast to query and indexable, unlike `json` which stores raw text. The core operators: `->` extracts a field/element as JSONB, `->>` extracts it as text, `@>` tests containment (does the left value contain the right), and `jsonb_path_query` / the `@?` and `@@` operators evaluate SQL/JSON path expressions. The **why** for the two arrows is type: use `->>` when you need to compare to a text value, `->` when you keep navigating deeper.

The strategic answer to "JSONB vs columns" is: use real columns for structured, queried, constrained data because they get statistics, precise types, and cheap indexes. Reach for JSONB when the shape is **variable, sparse, or genuinely schema-less** (user-defined attributes, third-party payloads, event blobs). Mixing both is common, promote hot fields to columns and keep the tail in JSONB.

A GIN index makes containment and existence queries fast. The default `jsonb_ops` supports `@>`, `?`, etc.; `jsonb_path_ops` is smaller and faster for `@>` only.

```
CREATE INDEX ON docs USING gin (data jsonb_path_ops);

SELECT data -- containment uses the GIN index
FROM docs
WHERE data @> '{"status":"active"}'
  AND (data ->> 'age')::int > 30;
```

> **GOTCHA**
>
> Chaining `->` then comparing to a string silently fails: `data -> 'k' = 'x'` compares JSONB to text and errors or never matches. Use `data ->> 'k' = 'x'`. Also, a B-tree index on `(data ->> 'k')` can beat GIN for a single scalar field.

91 Full-Text Search

SEARCH

Q How does full-text search work with tsvector, tsquery, and ranking?

A Full-text search transforms documents into a normalized, searchable form. A `tsvector` is a sorted list of **lexemes** (normalized word roots) with positions, produced by `to_tsvector()` using a text-search configuration that handles stemming, stop words, and case folding. A `tsquery` is a normalized search expression built with `to_tsquery`, `plainto_tsquery`, or `websearch_to_tsquery`. The **why** is that matching lexemes rather than raw substrings gives you "running" matching "ran" and ignores noise words, far better than `LIKE '%...%'`.

Matching uses the `@@` operator between a tsvector and tsquery. A GIN index on the tsvector makes this fast at scale. Ranking with `ts_rank` or `ts_rank_cd` scores results by term frequency and proximity so you can order by relevance, and weights (A/B/C/D) let a title matter more than the body.

For performance and correctness, store or generate the tsvector as a column (a generated column is ideal) rather than recomputing it per query.

```
ALTER TABLE articles ADD COLUMN tsv tsvector
  GENERATED ALWAYS AS (to_tsvector('english', title || ' ' || body))
STORED;
CREATE INDEX ON articles USING gin (tsv);

SELECT id, ts_rank(tsv, q) AS rank
FROM articles, websearch_to_tsquery('english', 'postgres index') q
WHERE tsv @@ q ORDER BY rank DESC;
```

> **GOTCHA**
>
> The text-search configuration must match at index and query time. Indexing with `'english'` but querying with `'simple'` yields inconsistent lexemes and the index may not be used. Pin the configuration explicitly on both sides.

92 Useful Extensions

ECOSYSTEM

Q What are some essential PostgreSQL extensions and what do they provide?

A Extensions are packaged add-ons installed with `CREATE EXTENSION` that add types, functions, and index support without recompiling the server. This modularity is a hallmark of PostgreSQL, the core stays lean while specialized capabilities plug in. Most ship in the `contrib` package.

`pg_stat_statements` is the single most valuable observability extension: it aggregates normalized query statistics (calls, total/mean time, rows, I/O) so you can find your slowest and most frequent queries. `pgcrypto` provides hashing and symmetric/asymmetric encryption functions like `crypt()` and `digest()`. `uuid-ossp` (or the built-in `gen_random_uuid()`) generates UUIDs. `PostGIS` turns PostgreSQL into a full geospatial database with geometry/geography types, spatial indexes, and hundreds of GIS functions.

Extensions can be versioned and upgraded with `ALTER EXTENSION ... UPDATE`, and some require being listed in `shared_preload_libraries` (and a restart) because they hook into the server, pg_stat_statements is one such.

```
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
CREATE EXTENSION IF NOT EXISTS pgcrypto;

SELECT query, calls, mean_exec_time
FROM pg_stat_statements
ORDER BY total_exec_time DESC LIMIT 10;
```

> **GOTCHA**
>
> Since PostgreSQL 13+ you rarely need `uuid-ossp`, the built-in `gen_random_uuid()` covers UUID v4. Adding an extension just for that pulls in an unnecessary dependency.

93 Functions and Stored Procedures

PROGRAMMING

Q What is the difference between functions and procedures in PL/pgSQL?

A PL/pgSQL is PostgreSQL's procedural language for writing server-side logic with variables, loops, conditionals, and exception handling. The important distinction: a `FUNCTION` returns a value and runs **inside** a surrounding transaction, so it cannot commit or roll back. A `PROCEDURE` (added in PostgreSQL 11) is invoked with `CALL`, does not return a value the same way, and crucially **can manage transactions** with `COMMIT` and `ROLLBACK` mid-body.

This is why procedures exist: for batch jobs that must commit in chunks (e.g., processing millions of rows without holding one giant transaction). Functions are for computations, derived values, and anything used in a `SELECT`. Both support `SECURITY DEFINER` to run with the definer's privileges, useful but a common security footgun if the search_path is not pinned.

Exception blocks (`EXCEPTION WHEN ... THEN`) provide savepoint-based error handling; note that catching an exception has real cost because it establishes a subtransaction.

```
CREATE PROCEDURE purge_batches()
LANGUAGE plpgsql AS $$
BEGIN
  LOOP
    DELETE FROM logs WHERE id IN
      (SELECT id FROM logs WHERE old LIMIT 1000);
    EXIT WHEN NOT FOUND;
    COMMIT;  -- only legal in a procedure
  END LOOP;
END; $$;
```

GOTCHA

A `SECURITY DEFINER` function that does not set `search_path` can be hijacked: a caller creating a same-named object in a schema earlier on the path can make your function call their code with elevated rights. Always `SET search_path` on such functions.

94 Triggers

PROGRAMMING

Q How do triggers work, and what are BEFORE, AFTER, and row vs statement level?

A A trigger binds a function to a table event (INSERT/UPDATE/DELETE/TRUNCATE) so custom logic runs automatically. The **timing** matters: a `BEFORE` trigger fires before the row is written and can **modify or veto** the row by returning a changed `NEW` or `NULL` to skip it, making it the place for validation and derived-column population. An `AFTER` trigger fires once the change is durable and sees the final state, ideal for auditing, cascading to other tables, or notifications.

The **level** also matters: a `FOR EACH ROW` trigger runs once per affected row and has access to `OLD` / `NEW`; a `FOR EACH STATEMENT` trigger runs once per statement regardless of row count, useful for logging that an operation happened. Trigger functions return type `trigger` and are written in PL/pgSQL or other languages.

Triggers are powerful but can hide behavior and hurt bulk-operation performance since row-level triggers fire per row.

```
CREATE FUNCTION set_updated_at() RETURNS trigger AS $$
BEGIN
  NEW.updated_at := now();
  RETURN NEW;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER t_updated BEFORE UPDATE ON accounts
  FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

> **GOTCHA**
>
> Multiple triggers on the same event fire in **alphabetical order by name**, not creation order. If one trigger depends on another's effect, name them carefully. Also, `TRUNCATE` does not fire row-level triggers, only statement-level ones.

95 Views vs Materialized Views

PERFORMANCE

Q What is the difference between a view and a materialized view, and how do you refresh?

A A regular `VIEW` is a stored query, it holds no data and is expanded into the referencing query at runtime, so it always reflects current data but pays the full computation cost on every access. A `MATERIALIZED VIEW` physically stores the result set on disk, so reads are fast, but the data is a **snapshot** that goes stale until you refresh it. The trade-off is freshness versus read cost.

You refresh with `REFRESH MATERIALIZED VIEW`, which recomputes the whole query and replaces the contents, taking an `ACCESS EXCLUSIVE` lock that blocks readers. To avoid blocking, use `REFRESH ... CONCURRENTLY`, which requires a **unique index** on the materialized view and updates rows in place while readers continue.

Use materialized views for expensive aggregations or reporting rollups that tolerate some staleness. For always-fresh results, use a regular view (optionally with good indexes on the base tables) or maintain a summary table via triggers.

```
CREATE MATERIALIZED VIEW daily_sales AS
  SELECT day, sum(amount) AS total
  FROM orders GROUP BY day;

CREATE UNIQUE INDEX ON daily_sales (day);
REFRESH MATERIALIZED VIEW CONCURRENTLY daily_sales;
```

> **GOTCHA**
>
> A newly created materialized view is populated immediately unless you use `WITH NO DATA`, in which case it is unscannable until the first `REFRESH`. And `CONCURRENTLY` is impossible without a unique index, plan for it up front.

96 Sequences and Identity Columns

SCHEMA DESIGN

Q How do sequences relate to SERIAL, IDENTITY, and generated columns?

A A `SEQUENCE` is a special object that hands out monotonically increasing numbers atomically, even across concurrent transactions. Historically the `SERIAL` pseudo-type auto-created a sequence and set a column default of `nextval()`. The modern, SQL-standard replacement is `GENERATED ... AS IDENTITY`, which is preferred because it ties the sequence tightly to the column and prevents accidental manual inserts (with `ALWAYS`).

`GENERATED ALWAYS AS IDENTITY` forbids user-supplied values (you must use `OVERRIDING SYSTEM VALUE` to force one), while `GENERATED BY DEFAULT` allows them, which is what you want when restoring data with explicit keys. This is cleaner than SERIAL, whose sequence ownership and permissions were easy to get wrong.

Separately, a **generated column** (`GENERATED ALWAYS AS (expr) STORED`) is a different feature: it computes a value from other columns in the same row, not from a sequence. Sequences are non-transactional: gaps are normal because a rolled-back transaction does not return its consumed number.

```
CREATE TABLE users (
  id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  email text,
  email_lower text GENERATED ALWAYS AS (lower(email)) STORED
);
```

> **GOTCHA**
>
> Never assume sequence values are gap-free or reflect insertion order across concurrent sessions. Rollbacks, caching (`CACHE`), and crashes all create gaps. Treat them as unique identifiers, not row counts.

97 Preventing SQL Injection

SECURITY

Q How do parameterized queries prevent SQL injection, and what are the pitfalls?

A SQL injection happens when untrusted input is concatenated into a query string so that attacker-supplied text is parsed as SQL rather than treated as data. The definitive defense is **parameterized queries** (prepared statements): the SQL text with placeholders is sent and planned separately from the parameter values, so values can never change the query's structure. The **why** is that the parser sees the query before the data exists, the data is bound afterward and is always a literal.

In PostgreSQL the wire protocol uses numbered placeholders (`$1`, `$2`); client libraries expose this as parameter binding. Escaping input manually is error-prone and should be a last resort. Note that parameters can only stand in for **values**, not identifiers (table/column names) or keywords, injecting those safely requires an allowlist or `quote_ident()`.

Inside PL/pgSQL, dynamic SQL built with `EXECUTE` should use `USING` for values and `format()` with `%I` / `%L` for identifiers/literals.

```
-- application: value is bound, never parsed as SQL
SELECT * FROM users WHERE email = $1;

-- dynamic SQL in PL/pgSQL, done safely
EXECUTE format('SELECT * FROM %I WHERE id = $1', tbl)
  USING user_id;
```

> **GOTCHA**
>
> A parameter cannot replace a table or column name. Code that does `format('... ORDER BY %s', col)` with user input is injectable, use `%I` (`quote_ident`) or validate against a fixed allowlist of column names.

98 Normalization vs Denormalization

SCHEMA DESIGN

Q What are the trade-offs between normalization and denormalization?

A Normalization organizes data to eliminate redundancy, each fact is stored once, with relationships expressed via foreign keys. The payoff is **data integrity**: there is a single source of truth, so updates cannot leave inconsistent copies, and anomalies on insert/update/delete are avoided. The cost is that reconstructing a full picture requires JOINs, which add query complexity and can be slower.

Denormalization deliberately introduces redundancy, duplicating or pre-aggregating data, to make reads faster by avoiding joins or repeated computation. The trade-off reverses: you gain read performance but take on the burden of keeping the duplicated data consistent (via triggers, application logic, or periodic rebuilds), and you use more storage.

The pragmatic stance: **normalize by default** for correctness, then denormalize selectively where profiling shows a real read bottleneck. PostgreSQL offers middle grounds, materialized views, generated columns, and JSONB, that give some denormalization benefits while keeping the base tables clean.

```
-- normalized: order_items reference products
SELECT o.id, sum(oi.qty * p.price)
FROM orders o
JOIN order_items oi ON oi.order_id = o.id
JOIN products p ON p.id = oi.product_id
GROUP BY o.id;
```

> **GOTCHA**
>
> Premature denormalization is a classic mistake, it optimizes a read cost you have not measured while permanently adding write-consistency risk. Prove the bottleneck with `EXPLAIN ANALYZE` first; a proper index often removes the need to denormalize at all.

99 Monitoring and Key Metrics

OPERATIONS

Q What are the key metrics for monitoring PostgreSQL, and how do you read pg_stat_activity?

A PostgreSQL exposes rich statistics through system views, so effective monitoring is mostly about knowing which to query. `pg_stat_activity` shows one row per connection: its `state` (active, idle, idle in transaction), the current `query`, `wait_event`, and how long it has been running. This is your first stop for finding stuck queries, blocking, or connection exhaustion.

The **cache hit ratio** measures what fraction of block reads were served from the shared buffer cache versus disk, computed from `pg_stat_database` as `blks_hit / (blks_hit + blks_read)`. A healthy OLTP system is typically well above 99%; a sudden drop signals memory pressure or a query scanning far more data than expected. Other key signals: replication lag, transaction ID age (to avoid wraparound), long-running `idle in transaction` sessions (which block vacuum and hold locks), and per-query time from `pg_stat_statements`.

Combine these with dead-tuple counts in `pg_stat_user_tables` to know when autovacuum is falling behind.

```
-- find long-running / blocking sessions
SELECT pid, state, now() - query_start AS runtime, query
FROM pg_stat_activity
WHERE state != 'idle' ORDER BY runtime DESC;

-- cache hit ratio
SELECT sum(blks_hit)::float / sum(blks_hit + blks_read)
FROM pg_stat_database;
```

GOTCHA

Idle in transaction sessions are silent killers: they hold locks and a snapshot that prevents vacuum from cleaning dead rows, causing table bloat. Set `idle_in_transaction_session_timeout` to terminate them automatically.

100 PostgreSQL vs MySQL

COMPARISON

Q What are the key differences between PostgreSQL and MySQL?

A Both are mature open-source relational databases, but their philosophies differ. PostgreSQL positions itself as a standards-compliant, extensible **object-relational** system emphasizing correctness and advanced features; MySQL historically prioritized simplicity and raw read speed for web workloads. A structural difference is the storage engine: MySQL has pluggable engines (InnoDB, MyISAM), whereas PostgreSQL has a single, tightly-integrated engine.

Feature-wise, PostgreSQL has long led on rich types and capabilities: native JSONB with indexing, arrays, ranges, full-text search, table inheritance, transactional DDL (you can `ROLLBACK` a `CREATE TABLE`), a wide range of index types (GIN, GiST, BRIN, partial, expression), and materialized views. Its MVCC keeps old row versions in the table, requiring **VACUUM**. MySQL/InnoDB stores row versions in a separate undo log and is often praised for simple replication setup.

On standards and extensibility PostgreSQL wins (custom types, extensions like PostGIS, procedural languages); MySQL retains a large hosting ecosystem and familiarity. In practice, choose PostgreSQL for complex queries, data integrity, and advanced features, MySQL where its ecosystem or specific tooling fits.

```
-- transactional DDL: PostgreSQL can roll this back
BEGIN;
CREATE TABLE t (id int);
ROLLBACK;  -- table is gone; MySQL would have auto-committed
```

> **GOTCHA**
>
> A frequent surprise for MySQL migrants: PostgreSQL is strict about types and does not silently coerce or truncate bad data by default, and identifiers fold to **lowercase** (not uppercase) unless double-quoted. Silent-conversion habits from MySQL cause errors here, which is a feature, not a bug.

You made it to 100.

If you can explain most of these out loud, you are ready for almost any PostgreSQL interview. The candidates who stand out are not the ones who memorized syntax, they are the ones who can reason about how the planner picks an index and why MVCC lets readers and writers coexist. When you hit a question that is not here, walk it the same way: what problem does this solve, and what does it cost?

Found this useful?

This is part of a series of deep-dive interview guides. Follow for the next one, a new technology every few days.

Save this PDF and share it with someone who is interviewing.